Assessment and Answer Guide

DEVOPS ENGINEERING MASTERY — Volume 09, Companion Volume. First Edition, 9 November 2026. This guide contains 120 MCQs, 30 short-answer questions, 20 scenario-based troubleshooting questions, 10 incident case studies, the practical final examination and the capstone rubric — followed by a separate answer key with explanations. Attempt every question yourself before reading its answer; the explanation matters more than the letter.

> **WARNING:** Do not copy assignment solutions word for word. Read the outline, close this guide, and do the work yourself in your lab environment. That is where the learning happens.

মূল্যায়ন ও উত্তর নির্দেশিকা

মূল্যায়ন, উত্তর নির্দেশিকা ও কুইক রেফারেন্স — Ansible Automation

এই অংশটি **DevOps Engineering Mastery — Volume 09: Configuration Management & Cloud Provisioning Engineering (Ansible Automation)** বইয়ের মূল্যায়ন ও সহায়ক অংশ। এখানে আছে ১২০টি বহুনির্বাচনি প্রশ্ন (MCQ), ৩০টি সংক্ষিপ্ত প্রশ্ন, ২০টি পরিস্থিতিভিত্তিক সমস্যা-সমাধান প্রশ্ন, ১০টি প্রোডাকশন ইনসিডেন্ট কেস স্টাডি, একটি ব্যবহারিক চূড়ান্ত পরীক্ষা, সম্পূর্ণ উত্তর নির্দেশিকা, ১৫টি ব্যবহারিক অ্যাসাইনমেন্ট, ১৬০টি পদের শব্দকোষ (Glossary), Ansible CLI, inventory, playbook, module, variable, Jinja2 ও Vault রেফারেন্স, চেকলিস্ট, ট্রাবলশুটিং ডিসিশন ট্রি, দক্ষতা-ট্র্যাকিং শিট এবং অফিসিয়াল রেফারেন্স। চলমান উদাহরণ প্রতিষ্ঠান **TechNova** এবং MERN স্ট্যাক (MongoDB, Express, React, Next.js, Node.js) প্রসঙ্গ এই পুরো অংশে ব্যবহৃত হয়েছে। পরিবেশ: Ubuntu 24.04, Ansible Core 2.17+, কাল্পনিক ডোমেইন `technova.example`, TEST-NET আইপি `203.0.113.x`।

> **সতর্কতা — আগে চেষ্টা, তারপর উত্তর:** উত্তর নির্দেশিকা পড়ার আগে প্রতিটি প্রশ্ন নিজে চেষ্টা করুন। MCQ-এর অক্ষর মুখস্থ করলে কিছু শিখবেন না — ব্যাখ্যাটি বুঝুন, তারপর ল্যাবে নিজের playbook-এ প্রয়োগ করুন। চূড়ান্ত পরীক্ষাটি সময় ধরে, নোট বন্ধ রেখে দেওয়ার চেষ্টা করুন।

> **কীভাবে ব্যবহার করবেন:** প্রতিটি অধ্যায় পড়ার পর সংশ্লিষ্ট MCQ ব্লকটি সমাধান করুন। ভুল হওয়া প্রশ্নের অধ্যায়টি আবার পড়ুন এবং ল্যাব ওয়ার্কবুকের সংশ্লিষ্ট ল্যাবটি করুন। কঠিনতা স্তর দেখে এগোন — শুরুর স্তর (Beginner) দিয়ে শুরু করুন, তারপর মধ্যম (Intermediate), সবশেষে উন্নত (Advanced)।

কঠিনতা স্তর

স্তর	অর্থ	কার জন্য
শুরুর স্তর (Beginner)	মৌলিক ধারণা, কমান্ড ও সিনট্যাক্স	Ansible-এ নতুন শিক্ষার্থী
মধ্যম স্তর (Intermediate)	বাস্তব প্রয়োগ, যুক্তি ও সিদ্ধান্ত	playbook লিখতে পারে এমন শিক্ষার্থী

উন্নত স্তর (Advanced)	জটিল নকশা, ট্রাবলশুটিং ও প্রোডাকশন	প্রোডাকশন পরিবেশে কাজ করতে প্রস্তুত শিক্ষার্থী

মূল্যায়ন কাঠামো এক নজরে

অংশ	পরিমাণ	উদ্দেশ্য
MCQ (অংশ A)	১২০টি	২৪টি অধ্যায়ের ধারণাগত বোঝাপড়া যাচাই
সংক্ষিপ্ত প্রশ্ন (অংশ B)	৩০টি	সংক্ষেপে ব্যাখ্যা করার ক্ষমতা
পরিস্থিতিভিত্তিক প্রশ্ন (অংশ C)	২০টি	বাস্তব সমস্যায় নির্ণয় ও সিদ্ধান্ত
ইনসিডেন্ট কেস স্টাডি (অংশ D)	১০টি	প্রোডাকশন ঘটনা থেকে শিক্ষা
চূড়ান্ত পরীক্ষা (অংশ E)	১০০ নম্বর, ৩ ঘণ্টা	তত্ত্ব + হাতে-কলমে MERN ডেপ্লয়মেন্ট
উত্তর নির্দেশিকা (অংশ F)	সব প্রশ্নের উত্তর ও ব্যাখ্যা	স্ব-মূল্যায়ন, রুব্রিক ও ১৫টি অ্যাসাইনমেন্ট
শব্দকোষ (অংশ G)	১৬০টি পদ	বাংলা ব্যাখ্যাসহ প্রযুক্তিগত শব্দভাণ্ডার
কুইক রেফারেন্স (অংশ H–K)	রেফারেন্স, চেকলিস্ট, ডিসিশন ট্রি, ট্র্যাকিং শিট	কাজের পাশে রাখার সহায়ক

নিয়ম: এই বইয়ের সব উদাহরণে কাল্পনিক ডোমেইন (`technova.example`), TEST-NET আইপি (`203.0.113.x`) এবং কাল্পনিক প্লেসহোল্ডার (`CHANGE-ME-...`) ব্যবহার করা হয়েছে। কোনো আসল secret, token, পাসওয়ার্ড বা মূল্য এই বইয়ে নেই। সব নমুনা টার্মিনাল আউটপুট চিত্রণমূলক। TechNova-র সব সার্ভিস, হোস্টনেম ও ঠিকানা কাল্পনিক।

২৪টি অধ্যায়ের মানচিত্র

অধ্যায় ১ Configuration Management-এর ভূমিকা; ২ Ansible পরিচিতি ও আর্কিটেকচার; ৩ ইনস্টলেশন ও কন্ট্রোল নোড; ৪ inventory ভিত্তি; ৫ ad-hoc কমান্ড; ৬ playbook ভিত্তি; ৭ YAML ও playbook গঠন; ৮ variable ও precedence; ৯ facts ও conditionals; ১০ loop ও Jinja2; ১১ handler ও idempotency; ১২ role ও Galaxy; ১৩ collection; ১৪ Ansible Vault; ১৫ become ও privilege escalation; ১৬ dynamic inventory ও ক্লাউড; ১৭ hardening ও নিরাপত্তা; ১৮ MERN অ্যাপ ডেপ্লয়মেন্ট; ১৯ ক্লাউড প্রভিশনিং; ২০ Docker ও Kubernetes কনফিগারেশন; ২১ testing (Molecule, ansible-lint); ২২ AWX ও Tower; ২৩ governance, CI/CD ও GitOps; ২৪ Terraform ইন্টিগ্রেশন ও ক্যাপস্টোন।

অংশ A — বহুনির্বাচনি প্রশ্ন

১২০টি বহুনির্বাচনি প্রশ্ন (MCQ)

প্রতিটি প্রশ্নের চারটি বিকল্প আছে — ক, খ, গ, ঘ। সঠিক উত্তরটি বেছে নিন। উত্তর ও ব্যাখ্যা আছে অংশ F-এ, তবে আগে নিজে চেষ্টা করুন। প্রশ্নগুলো শুধু মুখস্থ পরীক্ষা করে না — কেন একটি সিদ্ধান্ত নেওয়া হয়, কোনো ব্যর্থতার কারণ কী, এবং কোন পদ্ধতি কখন উপযুক্ত, তা যাচাই করে।

ব্লক ১ — প্রশ্ন ১–২০: Configuration Management, Ansible পরিচিতি, ইনস্টলেশন, inventory (অধ্যায় ১–৪)

1. [শুরুর স্তর] Configuration Management-এর মূল উদ্দেশ্য কী?
   ক) সার্ভারের কনফিগারেশন একরূপ, পুনরাবৃত্তিযোগ্য ও সংস্করণ-নিয়ন্ত্রিত রাখা খ) সার্ভার মুছে ফেলা গ) শুধু লগ দেখা ঘ) নেটওয়ার্ক বন্ধ করা
2. [শুরুর স্তর] Ansible-এর সবচেয়ে বড় বৈশিষ্ট্য কোনটি?
   ক) Managed node-এ কোনো agent ইনস্টল করতে হয় না, SSH-এর মাধ্যমে কাজ করে খ) শুধু Windows-এ চলে গ) সবসময় agent লাগে ঘ) শুধু GUI দিয়ে চলে
3. [শুরুর স্তর] Ansible-এ control node বলতে কী বোঝায়?
   ক) যে মেশিন থেকে Ansible কমান্ড চালানো হয় খ) যে মেশিনে ডেটাবেস চলে গ) যে মেশিনে শুধু MongoDB চলে ঘ) ক্লাউডের বিলিং সার্ভার
4. [শুরুর স্তর] Ansible কোন ভাষায় লেখা?
   ক) Java খ) Python গ) Go ঘ) C++
5. [মধ্যম স্তর] Ansible-কে "agentless" বলা হয় কেন?
   ক) এতে কোনো প্রোগ্রাম লাগে না খ) Managed node-এ শুধু Python ও SSH দরকার, আলাদা agent সফটওয়্যার ইনস্টল করতে হয় না গ) এটি ইন্টারনেট ছাড়া চলে ঘ) এটি কোনো কনফিগারেশন করে না
6. [শুরুর স্তর] Inventory ফাইলের কাজ কী?
   ক) Managed host-গুলোর তালিকা ও তাদের গ্রুপ নির্ধারণ করা খ) পাসওয়ার্ড সংরক্ষণ করা গ) লগ সংরক্ষণ করা ঘ) কোড কম্পাইল করা
7. [শুরুর স্তর] INI inventory-তে গ্রুপ লেখার সঠিক উপায় কোনটি?
   ক) `group webservers` খ) `[webservers]` গ) `<webservers>` ঘ) `webservers:`
8. [শুরুর স্তর] `ansible all --list-hosts` কমান্ড কী করে?
   ক) সব হোস্টে ping করে খ) inventory-তে থাকা সব হোস্টের তালিকা দেখায় গ) সব হোস্ট মুছে ফেলে ঘ) playbook চালায়
9. [মধ্যম স্তর] TechNova-র web সার্ভার `203.0.113.10` ও db সার্ভার `203.0.113.20`। inventory-তে দুটি গ্রুপে ভাগ করার সুবিধা কী?
   ক) কোনো সুবিধা নেই খ) একই playbook দিয়ে ভিন্ন ভিন্ন টাস্ক ভিন্ন গ্রুপে চালানো যায় গ) সার্ভার দ্রুত হয় ঘ) IP বদলে যায়
10. [শুরুর স্তর] Ubuntu 24.04-এ Ansible ইনস্টল করার সুপারিশকৃত উপায় কোনটি?
    ক) `pip install ansible` বা `apt install ansible` (অফিসিয়াল পদ্ধতি অনুযায়ী) খ) সোর্স কোড হাতে কপি করা গ) Windows installer চালানো ঘ) `npm install ansible`

11. [মধ্যম স্তর] `ansible.cfg` ফাইলের কাজ কী?
    ক) Ansible-এর ডিফল্ট সেটিং (inventory পথ, remote user, host key checking ইত্যাদি) নির্ধারণ করা খ) playbook লেখা গ) পাসওয়ার্ড এনক্রিপ্ট করা ঘ) সার্ভার তৈরি করা
12. [মধ্যম স্তর] `host_key_checking = False` সেট করা কেন ঝুঁকিপূর্ণ?
    ক) এটি Ansible ধীর করে খ) এটি man-in-the-middle আক্রমণের ঝুঁকি বাড়ায়, কারণ হোস্টের পরিচয় যাচাই হয় না গ) এটি SSH বন্ধ করে দেয় ঘ) এটি inventory মুছে ফেলে
13. [শুরুর স্তর] YAML inventory-তে হোস্ট লেখার সঠিক উদাহরণ কোনটি?
    ক) `web1: ansible_host: 203.0.113.10` খ) `all: hosts: web1: ansible_host: 203.0.113.10` গ) `web1 = 203.0.113.10` ঘ) `<web1>203.0.113.10</web1>`
14. [মধ্যম স্তর] `ansible_host` variable-এর কাজ কী?
    ক) হোস্টের আসল সংযোগ ঠিকানা (IP বা DNS) নির্ধারণ করা, যা inventory নাম থেকে আলাদা হতে পারে খ) হোস্টের নাম বদলে দেওয়া গ) হোস্ট মুছে ফেলা ঘ) পোর্ট বন্ধ করা
15. [মধ্যম স্তর] একটি হোস্ট দুটি গ্রুপে থাকলে কী হয়?
    ক) ত্রুটি হয় খ) হোস্টটি দুই গ্রুপেরই variable পায়; precedence নিয়মে দ্বন্দ্ব মেটে গ) হোস্ট মুছে যায় ঘ) playbook চলে না
16. [উন্নত স্তর] `children` গ্রুপের ব্যবহার কী?
    ক) একটি গ্রুপের ভেতরে অন্য গ্রুপগুলোকে অন্তর্ভুক্ত করে meta-গ্রুপ তৈরি করা খ) হোস্ট মুছে ফেলা গ) পাসওয়ার্ড লুকানো ঘ) লগ ভাগ করা
17. [শুরুর স্তর] `ansible-inventory --graph` কী দেখায়?
    ক) গ্রুপ ও হোস্টের গাছ-আকৃতির কাঠামো খ) নেটওয়ার্কের গতি গ) CPU ব্যবহার ঘ) লগ ফাইল
18. [মধ্যম স্তর] SSH key-based authentication ব্যবহারের সুবিধা কী?
    ক) পাসওয়ার্ড টাইপ করতে হয় না এবং স্বয়ংক্রিয় কাজ নিরাপদে করা যায় খ) SSH বন্ধ হয়ে যায় গ) সার্ভার দ্রুত হয় ঘ) inventory ছোট হয়
19. [উন্নত স্তর] Inventory-তে `ansible_port` বদলে দেওয়া কখন দরকার হয়?
    ক) কখনোই নয় খ) SSH ডিফল্ট ২২ পোর্টের বদলে অন্য পোর্টে চললে গ) পাসওয়ার্ড বদলাতে ঘ) playbook বদলাতে
20. [মধ্যম স্তর] Configuration drift বলতে কী বোঝায়?
    ক) সার্ভারের কনফিগারেশন সময়ের সঙ্গে হাতে বদলে গিয়ে নির্ধারিত অবস্থা থেকে সরে যাওয়া খ) সার্ভার বন্ধ হয়ে যাওয়া গ) নেটওয়ার্ক ধীর হওয়া ঘ) ডিস্ক পূর্ণ হওয়া

ব্লক ২ — প্রশ্ন ২১–৪০: Ad-hoc কমান্ড, playbook ভিত্তি, YAML (অধ্যায় ৫–৭)

1. [শুরুর স্তর] Ad-hoc কমান্ড কখন ব্যবহার করা হয়?
    ক) একবারের ছোট কাজের জন্য, যেমন সব হোস্টে ping করা বা প্যাকেজ চেক করা খ) শুধু বড় ডেপ্লয়মেন্টে গ) কখনোই নয় ঘ) শুধু Vault-এ
2. [শুরুর স্তর] সব হোস্টে connectivity পরীক্ষার কমান্ড কোনটি?
    ক) `ansible all -m ping` খ) `ansible all -m pong` গ) `ansible all --ping` ঘ) `ping all`
3. [শুরুর স্তর] `ansible webservers -m command -a "uptime"` কী করে?
    ক) webservers গ্রুপের সব হোস্টে `uptime` কমান্ড চালায় খ) uptime মুছে ফেলে গ) webservers মুছে ফেলে ঘ) playbook তৈরি করে

4. [মধ্যম স্তর] `command` module আর `shell` module-এর পার্থক্য কী?
   ক) কোনো পার্থক্য নেই খ) `shell` পাইপ, রিডাইরেকশন ও shell variable বোঝে; `command` সরাসরি কমান্ড চালায়, shell feature বোঝে না গ) `command` দ্রুততর ঘ) `shell` শুধু Windows-এ চলে
5. [শুরুর স্তর] Playbook কী?
   ক) Ansible-এর কাজের ধাপগুলো YAML ফাইলে লেখা নির্দেশনার সেট খ) একটি বই গ) একটি ডেটাবেস ঘ) একটি লগ ফাইল
6. [শুরুর স্তর] Playbook চালানোর কমান্ড কোনটি?
   ক) `ansible-playbook site.yml` খ) `ansible run site.yml` গ) `playbook site.yml` ঘ) `ansible-play site.yml`
7. [শুরুর স্তর] YAML-এ indentation-এর নিয়ম কী?
   ক) যেকোনোভাবে লেখা যায় খ) Space দিয়ে ধারাবাহিক indentation দিতে হয়; tab ব্যবহার করা যায় না গ) শুধু tab ব্যবহার করতে হয় ঘ) indentation লাগে না
8. [মধ্যম স্তর] Play-তে `hosts:` কী নির্ধারণ করে?
   ক) কোন হোস্ট বা গ্রুপে play-টি চলবে খ) পাসওয়ার্ড গ) পোর্ট ঘ) লগের জায়গা
9. [মধ্যম স্তর] Task-এ `name:` লেখার সুবিধা কী?
   ক) আউটপুটে ধাপটি পড়া যায় ও ডিবাগ করা সহজ হয় খ) Task দ্রুত হয় গ) Task নিরাপদ হয় ঘ) কোনো সুবিধা নেই
10. [শুরুর স্তর] `ansible-playbook --check` কী করে?
    ক) আসলে কিছু না বদলে কী বদলাতো তা দেখায় (dry run) খ) playbook মুছে ফেলে গ) playbook চালায় ঘ) inventory মুছে ফেলে
11. [মধ্যম স্তর] `--limit webservers` flag-এর কাজ কী?
    ক) playbook শুধু নির্দিষ্ট হোস্ট/গ্রুপে চালানো খ) playbook বন্ধ করা গ) গতি বাড়ানো ঘ) লগ বন্ধ করা
12. [মধ্যম স্তর] `--tags` দিয়ে কী করা যায়?
    ক) শুধু নির্দিষ্ট tag-যুক্ত task চালানো বা বাদ দেওয়া খ) tag মুছে ফেলা গ) inventory বদলানো ঘ) Vault খোলা
13. [উন্নত স্তর] Playbook-এ `gather_facts: false` দিলে কী হয়?
    ক) playbook ব্যর্থ হয় খ) fact সংগ্রহের ধাপটি বাদ যায়, ফলে play দ্রুত শুরু হয় গ) সব variable মুছে যায় ঘ) হোস্ট সংযোগ বিচ্ছিন্ন হয়
14. [মধ্যম স্তর] YAML-এ list লেখার সঠিক উপায় কোনটি?
    ক) `- item1` (ড্যাশ দিয়ে) খ) `(item1)` গ) `[item1]` শুধু ঘ) `item1;`
15. [মধ্যম স্তর] `ansible-playbook --syntax-check site.yml` কী করে?
    ক) playbook-এর YAML ও সিনট্যাক্স যাচাই করে, না চালিয়ে খ) playbook চালায় গ) সার্ভার তৈরি করে ঘ) Vault খোলে
16. [শুরুর স্তর] `become: true` play-তে দিলে কী হয়?
    ক) সব task root (sudo) হিসেবে চলে খ) playbook দ্রুত হয় গ) হোস্ট বদলে যায় ঘ) কিছুই হয় না
17. [মধ্যম স্তর] একটি playbook-এ একাধিক play থাকলে কী হয়?
    ক) ত্রুটি হয় খ) play-গুলো ক্রমানুসারে চলে, প্রতিটি ভিন্ন হোস্টে বা ভিন্ন কাজে গ) শুধু প্রথমটি চলে ঘ) শুধু শেষটি চলে

18. [উন্নত স্তর] `ansible-playbook --start-at-task "Install nginx"` কখন কাজে লাগে?
    ক) ব্যর্থ playbook-এর মাঝখান থেকে আবার চালাতে খ) playbook মুছতে গ) inventory বদলাতে ঘ) Vault খুলতে
19. [মধ্যম স্তর] Ad-hoc কমান্ডে `-m apt -a "name=nginx state=present"` কী করে?
    ক) nginx প্যাকেজ ইনস্টল করে খ) nginx মুছে ফেলে গ) nginx বন্ধ করে ঘ) কিছুই করে না
20. [উন্নত স্তর] Playbook-এ `serial: 2` দিলে কী হয়?
    ক) একসঙ্গে ২টি হোস্টে ব্যাচে play চলে, rolling update সম্ভব হয় খ) playbook ২ বার চলে গ) ২টি task চলে ঘ) হোস্ট ২টি মুছে যায়

ব্লক ৩ — প্রশ্ন ৪১–৬০: Variable, precedence, facts, conditionals (অধ্যায় ৮–৯)

1. [শুরুর স্তর] Variable-এর কাজ কী?
   ক) playbook-এ পুনঃব্যবহারযোগ্য মান সংরক্ষণ করা খ) সার্ভার মুছে ফেলা গ) লগ লুকানো ঘ) পোর্ট বন্ধ করা
2. [শুরুর স্তর] Playbook-এ variable লেখার সঠিক উপায় কোনটি?
   ক) `vars: app_port: 3000` খ) `var app_port = 3000` গ) `app_port == 3000` ঘ) `set app_port 3000`
3. [মধ্যম স্তর] `group_vars/webservers.yml` ফাইলের কাজ কী?
   ক) webservers গ্রুপের সব হোস্টের জন্য variable নির্ধারণ করা খ) webservers মুছে ফেলা গ) লগ সংরক্ষণ করা ঘ) পাসওয়ার্ড এনক্রিপ্ট করা
4. [মধ্যম স্তর] `host_vars/web1.yml` কখন ব্যবহার করা হয়?
   ক) শুধু web1 হোস্টের জন্য নির্দিষ্ট variable দিতে খ) সব হোস্টের জন্য গ) inventory মুছতে ঘ) কখনোই নয়
5. [উন্নত স্তর] Ansible variable precedence-এ সবচেয়ে শক্তিশালী কোনটি?
   ক) role defaults খ) কমান্ড লাইনের `-e` extra vars গ) group_vars ঘ) inventory variable
6. [উন্নত স্তর] Role defaults আর role vars-এর মধ্যে কোনটি জয়ী হয়?
   ক) role defaults খ) role vars (role vars-এর precedence বেশি) গ) দুটো সমান ঘ) কোনোটিই কাজ করে না
7. [শুরুর স্তর] Fact বলতে কী বোঝায়?
   ক) Ansible-এর সংগ্রহ করা হোস্টের তথ্য, যেমন OS, IP, মেমরি খ) একটি গল্প গ) একটি পাসওয়ার্ড ঘ) একটি লগ লাইন
8. [শুরুর স্তর] `ansible_facts['os_family']` কী দেয়?
   ক) হোস্টের OS পরিবার, যেমন Debian বা RedHat খ) হোস্টের নাম গ) হোস্টের পাসওয়ার্ড ঘ) হোস্টের গতি
9. [মধ্যম স্তর] `ansible all -m setup` কী করে?
   ক) সব হোস্টের fact সংগ্রহ করে দেখায় খ) সব হোস্টে সফটওয়্যার ইনস্টল করে গ) সব হোস্ট বন্ধ করে ঘ) inventory তৈরি করে
10. [মধ্যম স্তর] `when: ansible_facts['os_family'] == "Debian"` কী করে?
    ক) শুধু Debian পরিবারের হোস্টে task চালায় খ) সব হোস্টে চালায় গ) task মুছে ফেলে ঘ) OS বদলে দেয়
11. [মধ্যম স্তর] `when` শর্তে একাধিক শর্ত দিলে কী হয়?
    ক) সব শর্ত সত্য হলেই task চলে (AND যুক্তি) খ) যেকোনো একটি সত্য হলেই চলে গ) শর্ত উপেক্ষা করা হয় ঘ) task ব্যর্থ হয়

12. [মধ্যম স্তর] `register: result` কী করে?
    ক) task-এর আউটপুট একটি variable-এ সংরক্ষণ করে খ) ব্যবহারকারী নিবন্ধন করে গ) লগ মুছে ফেলে ঘ) হোস্ট নিবন্ধন করে
13. [উন্নত স্তর] `when: result.changed` কখন সত্য হয়?
    ক) আগের task কিছু বদলালে খ) সবসময় গ) কখনোই নয় ঘ) শুধু ব্যর্থ হলে
14. [মধ্যম স্তর] `default` filter-এর কাজ কী?
    ক) variable না থাকলে বিকল্প মান দেওয়া, যেমন `{{ app_port | default(3000) }}` খ) variable মুছে ফেলা গ) variable এনক্রিপ্ট করা ঘ) variable লুকানো
15. [উন্নত স্তর] `hostvars['web1']['ansible_host']` কী দেয়?
    ক) web1 হোস্টের `ansible_host` মান, অন্য হোস্ট থেকে ব্যবহার করা যায় খ) সব হোস্টের তালিকা গ) web1-এর পাসওয়ার্ড ঘ) কিছুই নয়
16. [মধ্যম স্তর] `vars_prompt` কখন ব্যবহার করা হয়?
    ক) playbook চলার সময় ব্যবহারকারীর কাছ থেকে মান চাওয়া হয়, যেমন পাসওয়ার্ড খ) playbook লেখার সময় গ) inventory তৈরির সময় ঘ) কখনোই নয়
17. [উন্নত স্তর] একই variable-এর মান group_vars ও host_vars-এ ভিন্ন হলে কোনটি জয়ী হয়?
    ক) group_vars খ) host_vars (host-নির্দিষ্ট মানের precedence বেশি) গ) দুটো মিলিয়ে যায় ঘ) ত্রুটি হয়
18. [মধ্যম স্তর] `ansible_facts['default_ipv4']['address']` কী দেয়?
    ক) হোস্টের প্রধান IPv4 ঠিকানা খ) হোস্টের নাম গ) হোস্টের MAC ঘ) হোস্টের OS
19. [উন্নত স্তর] `set_fact` module-এর কাজ কী?
    ক) play চলাকালীন নতুন fact/variable তৈরি করা খ) fact মুছে ফেলা গ) OS বদলানো ঘ) হোস্ট যোগ করা
20. [মধ্যম স্তর] TechNova-র db হোস্টে শুধু MongoDB ইনস্টল করতে চাইলে সঠিক পদ্ধতি কোনটি?
    ক) সব হোস্টে ইনস্টল করা খ) `when: inventory_hostname in groups['dbservers']` বা db গ্রুপে play চালানো গ) হাতে SSH করে ইনস্টল করা ঘ) playbook মুছে ফেলা

ব্লক ৪ — প্রশ্ন ৬১–৮০: Loop, Jinja2, handler, idempotency, role, collection (অধ্যায় ১০–১৩)

1. [শুরুর স্তর] Loop-এর কাজ কী?
    ক) একই task ভিন্ন ভিন্ন মান দিয়ে বারবার চালানো খ) playbook থামানো গ) হোস্ট মুছে ফেলা ঘ) লগ লেখা
2. [শুরুর স্তর] `loop: [80, 443]` দিয়ে কী করা যায়?
    ক) পোর্ট ৮০ ও ৪৪৩ — দুটি মান দিয়ে একই task চালানো খ) পোর্ট বন্ধ করা গ) পোর্ট মুছে ফেলা ঘ) কিছুই নয়
3. [মধ্যম স্তর] Loop-এ বর্তমান মানটি কোন variable-এ থাকে?
    ক) `item` খ) `loop_var` গ) `value` ঘ) `current`
4. [মধ্যম স্তর] `loop_control: loop_var: pkg` কেন ব্যবহার করা হয়?
    ক) loop variable-এর নাম বদলে `pkg` করা, nested loop-এ দ্বন্দ্ব এড়াতে খ) loop বন্ধ করতে গ) গতি বাড়াতে ঘ) কোনো কারণ নেই
5. [শুরুর স্তর] Jinja2 template-এ variable লেখার সঠিক চিহ্ন কোনটি?
    ক) `{{ variable }}` খ) `[[ variable ]]` গ) `$variable` ঘ) `<variable>`
6. [শুরুর স্তর] Jinja2-এ শর্ত লেখার চিহ্ন কোনটি?
    ক) `{% if ... %} ... {% endif %}` খ) `{{ if }}` গ) `#if` ঘ) `if()`

7. [মধ্যম স্তর] `{{ app_port | int }}` filter কী করে?
   ক) মানটিকে পূর্ণসংখ্যায় রূপান্তর করে খ) মানটি মুছে ফেলে গ) মানটি এনক্রিপ্ট করে ঘ) মানটি লুকায়
8. [মধ্যম স্তর] `{{ "hello" | upper }}`-এর ফল কী?
   ক) `HELLO` খ) `hello` গ) `Hello` ঘ) ত্রুটি
9. [মধ্যম স্তর] `template` module কী করে?
   ক) Jinja2 template রেন্ডার করে remote হোস্টে ফাইল হিসেবে বসায় খ) ফাইল মুছে ফেলে গ) ফাইল কপি করে (রেন্ডার ছাড়া) ঘ) ফাইল ডাউনলোড করে
10. [মধ্যম স্তর] `copy` আর `template` module-এর পার্থক্য কী?
    ক) কোনো পার্থক্য নেই খ) `copy` ফাইল হুবহু কপি করে; `template` variable রেন্ডার করে বসায় গ) `copy` দ্রুততর ঘ) `template` শুধু লোকাল কাজ করে
11. [শুরুর স্তর] Handler কী?
    ক) শুধুমাত্র notify পেলে চলা বিশেষ task, যেমন সার্ভিস restart খ) সাধারণ task গ) একটি error ঘ) একটি variable
12. [মধ্যম স্তর] `notify: Restart nginx` কখন handler চালায়?
    ক) সবসময় খ) শুধু notify করা task-টি changed হলে, play-এর শেষে গ) play-এর শুরুতে ঘ) কখনোই নয়
13. [মধ্যম স্তর] Idempotency বলতে কী বোঝায়?
    ক) একই playbook বারবার চালালে প্রথমবারের পর আর কোনো বদল হয় না খ) playbook একবারই চালানো যায় গ) playbook ধীর হয় ঘ) playbook ব্যর্থ হয়
14. [উন্নত স্তর] কোন task idempotent নয়?
    ক) `apt: name=nginx state=present` খ) `shell: echo "x" >> /tmp/file` — প্রতিবার চালালে লাইন যোগ হয় গ) `copy: src=a dest=b` ঘ) `user: name=app state=present`
15. [মধ্যম স্তর] Handler একাধিকবার notify পেলে কতবার চলে?
    ক) প্রতিবার খ) play-এর শেষে মাত্র একবার গ) কখনোই নয় ঘ) দুইবার
16. [শুরুর স্তর] Role কী?
    ক) playbook-এর পুনঃব্যবহারযোগ্য, সুসংগঠিত অংশ (tasks, handlers, vars, templates আলাদা ডিরেক্টরিতে) খ) একটি ব্যবহারকারী গ) একটি পাসওয়ার্ড ঘ) একটি সার্ভার
17. [মধ্যম স্তর] Role-এর ডিরেক্টরি কাঠামোতে tasks কোথায় থাকে?
    ক) `roles/<name>/tasks/main.yml` খ) `roles/<name>/vars/` গ) `roles/<name>/files/` ঘ) inventory-তে
18. [মধ্যম স্তর] `ansible-galaxy init myrole` কী করে?
    ক) role-এর আদর্শ ডিরেক্টরি কাঠামো তৈরি করে খ) role ইনস্টল করে গ) role মুছে ফেলে ঘ) playbook চালায়
19. [মধ্যম স্তর] Ansible Galaxy কী?
    ক) community role ও collection খুঁজে পাওয়া ও ইনস্টল করার হাব খ) একটি ক্লাউড গ) একটি ডেটাবেস ঘ) একটি সার্ভার
20. [উন্নত স্তর] Collection আর role-এর পার্থক্য কী?
    ক) কোনো পার্থক্য নেই খ) collection হলো role, module, plugin ও playbook-এর বড় বিতরণ একক; role তার ভেতরের একটি অংশ হতে পারে গ) collection শুধু Windows-এ চলে ঘ) role শুধু Galaxy-তে থাকে

ব্লক ৫ — প্রশ্ন ৮১–১০০: Vault, become, dynamic inventory, hardening (অধ্যায় ১৪–১৭)

1. [শুরুর স্তর] Ansible Vault-এর কাজ কী?
   ক) পাসওয়ার্ড ও secret এনক্রিপ্ট করে সংরক্ষণ করা খ) সার্ভার লক করা গ) playbook লুকানো ঘ) লগ এনক্রিপ্ট করা
2. [শুরুর স্তর] Vault ফাইল তৈরি করার কমান্ড কোনটি?
   ক) `ansible-vault create secrets.yml` খ) `ansible-vault make secrets.yml` গ) `vault create secrets.yml` ঘ) `ansible create secrets.yml`
3. [মধ্যম স্তর] Vault-এনক্রিপ্ট করা ফাইল playbook-এ ব্যবহার করতে কোন flag দরকার?
   ক) `--ask-vault-pass` বা `--vault-password-file` খ) `--ask-pass` গ) `--become` ঘ) কোনো flag লাগে না
4. [মধ্যম স্তর] `ansible-vault encrypt_string 'mypassword' --name 'db_password'` কী করে?
   ক) একটি string এনক্রিপ্ট করে variable হিসেবে ব্যবহারের উপযোগী আউটপুট দেয় খ) পাসওয়ার্ড মুছে ফেলে গ) ফাইল তৈরি করে ঘ) Vault বন্ধ করে
5. [উন্নত স্তর] Vault পাসওয়ার্ড Git-এ commit করা কেন বিপজ্জনক?
   ক) এটি Git ধীর করে খ) পাসওয়ার্ড ফাঁস হলে সব secret উন্মুক্ত হয়ে যায় গ) Git ব্যর্থ হয় ঘ) কোনো ঝুঁকি নেই
6. [শুরুর স্তর] `become: true` আর `become_user: postgres` একসঙ্গে দিলে কী হয়?
   ক) task postgres ব্যবহারকারী হিসেবে চলে খ) task root হিসেবে চলে গ) task ব্যর্থ হয় ঘ) কিছুই হয় না
7. [মধ্যম স্তর] `become_method: su` কখন ব্যবহার করা হয়?
   ক) sudo-র বদলে su দিয়ে privilege বাড়াতে খ) সবসময় গ) কখনোই নয় ঘ) Windows-এ
8. [মধ্যম স্তর] `ansible_become_password` কোথায় রাখা উচিত?
   ক) playbook-এ খোলাখুলিভাবে খ) Vault-এনক্রিপ্ট করা variable-এ গ) inventory-তে খোলাখুলিভাবে ঘ) Git-এ খোলাখুলিভাবে
9. [শুরুর স্তর] Dynamic inventory কী?
   ক) ক্লাউড বা CMDB থেকে স্বয়ংক্রিয়ভাবে হোস্ট তালিকা তৈরি করা খ) হাতে লেখা inventory গ) একটি লগ ফাইল ঘ) একটি playbook
10. [মধ্যম স্তর] AWS EC2 dynamic inventory-তে কোন plugin ব্যবহার করা হয়?
    ক) `aws_ec2` inventory plugin খ) `aws_s3` গ) `ec2_static` ঘ) `cloud_init`
11. [উন্নত স্তর] Dynamic inventory-তে `keyed_groups` কী করে?
    ক) হোস্টের tag বা বৈশিষ্ট্য অনুযায়ী স্বয়ংক্রিয়ভাবে গ্রুপ তৈরি করে খ) হোস্ট মুছে ফেলে গ) পাসওয়ার্ড তৈরি করে ঘ) লগ ভাগ করে
12. [মধ্যম স্তর] `ansible-inventory -i inventory.aws_ec2.yml --graph` কী দেখায়?
    ক) dynamic inventory থেকে পাওয়া হোস্ট ও গ্রুপের গাছ খ) AWS বিল গ) EC2 দাম ঘ) কিছুই নয়
13. [শুরুর স্তর] Server hardening বলতে কী বোঝায়?
    ক) সার্ভারের আক্রমণ-ক্ষেত্র কমিয়ে নিরাপত্তা শক্ত করা খ) সার্ভার দ্রুত করা গ) সার্ভার বড় করা ঘ) সার্ভার মুছে ফেলা
14. [মধ্যম স্তর] SSH hardening-এ প্রথম পদক্ষেপ কোনটি?
    ক) root login নিষ্ক্রিয় করা ও password authentication বন্ধ করে key-only করা খ) SSH পোর্ট খুলে দেওয়া গ) সবাইকে root দেওয়া ঘ) firewall বন্ধ করা

15. [মধ্যম স্তর] UFW-তে শুধু SSH, HTTP ও HTTPS খোলার Ansible task-এ কোন module ব্যবহার করা হয়?
    ক) `ufw` module খ) `firewalld` module গ) `iptables` কমান্ড সরাসরি ঘ) `copy` module
16. [উন্নত স্তর] `lineinfile` module কেন idempotent উপায়ে sshd_config বদলাতে ভালো?
    ক) এটি নির্দিষ্ট লাইন খুঁজে বদলায়, না থাকলে যোগ করে, বারবার চালালেও একই ফল দেয় খ) এটি ফাইল মুছে ফেলে গ) এটি দ্রুততর ঘ) এটি shell ব্যবহার করে
17. [মধ্যম স্তর] Fail2ban-এর কাজ কী?
    ক) বারবার ভুল লগইন চেষ্টা করা IP-কে সাময়িকভাবে ব্লক করা খ) সার্ভার বন্ধ করা গ) লগ মুছে ফেলা ঘ) পাসওয়ার্ড তৈরি করা
18. [উন্নত স্তর] CIS benchmark অনুযায়ী hardening করতে Ansible-এ কোন পদ্ধতি ভালো?
    ক) হাতে একবার করে বদলানো খ) community role (যেমন dev-sec) বা নিজের role দিয়ে নিয়মগুলো কোড হিসেবে প্রয়োগ করা গ) firewall বন্ধ করা ঘ) কিছুই না করা
19. [মধ্যম স্তর] `authorized_key` module কী করে?
    ক) ব্যবহারকারীর `~/.ssh/authorized_keys`-এ SSH public key যোগ করে খ) private key তৈরি করে গ) পাসওয়ার্ড বদলায় ঘ) ব্যবহারকারী মুছে ফেলে
20. [উন্নত স্তর] Ansible দিয়ে unattended-upgrades চালু করার সুবিধা কী?
    ক) নিরাপত্তা প্যাচ স্বয়ংক্রিয়ভাবে ইনস্টল হয়, ফলে জানা দুর্বলতা কমে খ) সার্ভার বন্ধ হয় গ) আপডেট বন্ধ হয় ঘ) কোনো সুবিধা নেই

ব্লক ৬ — প্রশ্ন ১০১–১২০: MERN ডেপ্লয়মেন্ট, ক্লাউড, Docker/K8s, testing, AWX, governance, Terraform (অধ্যায় ১৮–২৪)

1. [মধ্যম স্তর] TechNova-র MERN স্ট্যাক ডেপ্লয় করতে Ansible-এ সাধারণত কোন ক্রম অনুসরণ করা হয়?
    ক) প্রথমে অ্যাপ, তারপর OS খ) OS প্রস্তুতি → Node.js/MongoDB ইনস্টল → কোড ডেপ্লয় → সার্ভিস চালু → nginx রিভার্স প্রক্সি গ) শুধু nginx ইনস্টল ঘ) শুধু কোড কপি
2. [মধ্যম স্তর] Node.js অ্যাপকে systemd service হিসেবে চালানোর সুবিধা কী?
    ক) সার্ভার রিস্টার্ট হলে অ্যাপ নিজে থেকে আবার চালু হয় ও লগ পরিচালনা সহজ হয় খ) অ্যাপ দ্রুত হয় গ) কোড ছোট হয় ঘ) কোনো সুবিধা নেই
3. [মধ্যম স্তর] `git` module দিয়ে কোড ডেপ্লয় করার সুবিধা কী?
    ক) নির্দিষ্ট branch/commit checkout করা যায় ও idempotent উপায়ে আপডেট করা যায় খ) কোড মুছে ফেলা যায় গ) GitHub বন্ধ করা যায় ঘ) কোনো সুবিধা নেই
4. [উন্নত স্তর] Zero-downtime deployment-এ Ansible-এ কোন কৌশল ব্যবহার করা হয়?
    ক) একবারে সব সার্ভার বন্ধ করা খ) `serial` দিয়ে rolling update ও health check করে ধাপে ধাপে সার্ভার বদলানো গ) সার্ভার মুছে ফেলা ঘ) playbook মুছে ফেলা
5. [মধ্যম স্তর] MongoDB-র connection string-এ পাসওয়ার্ড কীভাবে নিরাপদে দেওয়া উচিত?
    ক) playbook-এ খোলাখুলিভাবে খ) Vault-এনক্রিপ্ট করা variable থেকে template-এ রেন্ডার করে গ) Git-এ খোলাখুলিভাবে ঘ) ইমেইলে পাঠিয়ে
6. [শুরুর স্তর] Cloud provisioning বলতে কী বোঝায়?
    ক) ক্লাউডে VM, নেটওয়ার্ক ও স্টোরেজের মতো সম্পদ কোড দিয়ে তৈরি করা খ) ক্লাউড মুছে ফেলা গ) ক্লাউডের বিল দেখা ঘ) ক্লাউডে লগইন করা

7. [মধ্যম স্তর] Ansible-এ AWS EC2 instance তৈরি করতে কোন collection/module ব্যবহার করা হয়?
   ক) `amazon.aws.ec2_instance` খ) `aws_create` গ) `ec2_new` ঘ) `cloud_vm`
8. [উন্নত স্তর] Terraform আর Ansible একসঙ্গে ব্যবহারের যুক্তি কী?
   ক) Terraform infrastructure তৈরি করে, Ansible তার ভেতরে সফটওয়্যার কনফিগার করে — দুটি ভিন্ন স্তরের কাজ খ) দুটো একই কাজ করে, একটি বাদ দেওয়া যায় গ) Terraform Ansible-কে বদলে দেয় ঘ) Ansible Terraform-কে বদলে দেয়
9. [মধ্যম স্তর] Terraform-এর state ফাইল থেকে Ansible inventory তৈরি করার উপায় কোনটি?
   ক) `terraform-inventory` বা Terraform output থেকে dynamic inventory স্ক্রিপ্ট ব্যবহার করা খ) হাতে IP লিখে রাখা গ) state ফাইল মুছে ফেলা ঘ) কোনো উপায় নেই
10. [মধ্যম স্তর] Docker কনটেইনার Ansible দিয়ে পরিচালনা করতে কোন module ব্যবহার করা হয়?
    ক) `community.docker.docker_container` খ) `docker_run` গ) `container_new` ঘ) `shell` শুধু
11. [উন্নত স্তর] Kubernetes-এ Ansible কী করে?
    ক) Kubernetes নিজে চালায় খ) `kubernetes.core` collection দিয়ে manifest প্রয়োগ, namespace ও resource পরিচালনা করে গ) Kubernetes মুছে ফেলে ঘ) কিছুই করে না
12. [মধ্যম স্তর] `ansible-lint`-এর কাজ কী?
    ক) playbook ও role-এর best practice ও সিনট্যাক্স ত্রুটি পরীক্ষা করা খ) playbook চালানো গ) সার্ভার তৈরি করা ঘ) লগ দেখা
13. [মধ্যম স্তর] Molecule কীসের জন্য ব্যবহার করা হয়?
    ক) role টেস্ট করার জন্য — container/VM-এ role চালিয়ে যাচাই করা খ) playbook লেখার জন্য গ) Vault খোলার জন্য ঘ) inventory তৈরির জন্য
14. [উন্নত স্তর] `--check --diff` একসঙ্গে দিলে কী সুবিধা হয়?
    ক) কী বদলাতো তা diff আকারে দেখা যায়, আসলে কিছু না বদলে খ) playbook দ্রুত চলে গ) playbook মুছে যায় ঘ) কোনো সুবিধা নেই
15. [শুরুর স্তর] AWX কী?
    ক) Ansible-এর ওয়েব UI, REST API ও scheduling-সহ open-source অটোমেশন প্ল্যাটফর্ম খ) একটি ক্লাউড গ) একটি ডেটাবেস ঘ) একটি playbook
16. [মধ্যম স্তর] AWX-এ job template কী?
    ক) playbook, inventory ও credential একত্রে বেঁধে পুনরায় চালানোর উপযোগী সংজ্ঞা খ) একটি ইমেইল টেমপ্লেট গ) একটি লগ ফাইল ঘ) একটি সার্ভার
17. [মধ্যম স্তর] AWX-এ credential-এর কাজ কী?
    ক) SSH key, Vault পাসওয়ার্ড ও ক্লাউড key নিরাপদে সংরক্ষণ ও job-এ ব্যবহার করা খ) পাসওয়ার্ড Git-এ রাখা গ) লগইন বন্ধ করা ঘ) কোনো কাজ নেই
18. [উন্নত স্তর] GitOps পদ্ধতিতে Ansible playbook কোথায় থাকে?
    ক) শুধু লোকাল মেশিনে খ) Git repository-তে, যেখানে পরিবর্তন review ও audit হয় এবং CI/CD তা প্রয়োগ করে গ) সার্ভারের /tmp-এ ঘ) ইমেইলে
19. [মধ্যম স্তর] Playbook-এ `environment:` কী নির্ধারণ করে?
    ক) task চলার সময়ের environment variable, যেমন proxy সেটিং খ) সার্ভারের OS গ) inventory ঘ) Vault পাসওয়ার্ড

20. [উন্নত স্তর] TechNova-র প্রোডাকশন ডেপ্লয়মেন্টে সবচেয়ে গুরুত্বপূর্ণ নিরাপত্তা অভ্যাস কোনটি?
    ক) সব secret Vault-এ রাখা, least privilege ব্যবহার করা, --check দিয়ে যাচাই করা ও পরিবর্তনের audit trail রাখা
    খ) root পাসওয়ার্ড সবাইকে দেওয়া গ) firewall বন্ধ করা ঘ) Vault পাসওয়ার্ড Git-এ রাখা

অংশ B — সংক্ষিপ্ত প্রশ্ন

৩০টি সংক্ষিপ্ত প্রশ্ন

প্রতিটি প্রশ্নের উত্তর ৩–৫ বাক্যে লিখুন। মডেল উত্তর আছে অংশ F-এ।

1. [শুরুর স্তর] Agentless automation বলতে কী বোঝায় এবং Ansible কেন agentless?
2. [শুরুর স্তর] Inventory-তে গ্রুপ ব্যবহারের দুটি সুবিধা লিখুন।
3. [শুরুর স্তর] Ad-hoc কমান্ড আর playbook-এর মধ্যে পার্থক্য কী?
4. [শুরুর স্তর] YAML-এ indentation ভুল হলে কী হয়?
5. [মধ্যম স্তর] `command` module-এর বদলে `shell` module কখন ব্যবহার করবেন?
6. [মধ্যম স্তর] `--check` মোডের সীমাবদ্ধতা কী?
7. [মধ্যম স্তর] Variable precedence কেন জানা জরুরি? একটি উদাহরণ দিন।
8. [মধ্যম স্তর] Fact সংগ্রহ বন্ধ করলে (`gather_facts: false`) কী হারাবেন?
9. [মধ্যম স্তর] `register` ও `when` একসঙ্গে কীভাবে কাজ করে?
10. [মধ্যম স্তর] Jinja2 filter-এর দুটি উদাহরণ দিন এবং প্রতিটির কাজ লিখুন।
11. [মধ্যম স্তর] `copy` module কখন `template` module-এর চেয়ে ভালো?
12. [মধ্যম স্তর] Handler কেন play-এর শেষে চলে? এর সুবিধা কী?
13. [মধ্যম স্তর] Idempotency যাচাই করার একটি বাস্তব পদ্ধতি লিখুন।
14. [মধ্যম স্তর] Role-এর `defaults/` আর `vars/` ডিরেক্টরির পার্থক্য কী?
15. [মধ্যম স্তর] Ansible Galaxy থেকে role ইনস্টল করার ধাপগুলো লিখুন।
16. [মধ্যম স্তর] Vault পাসওয়ার্ড হারিয়ে গেলে কী হবে এবং তা এড়ানোর উপায় কী?
17. [মধ্যম স্তর] `become` আর `become_user`-এর পার্থক্য কী?
18. [উন্নত স্তর] Dynamic inventory static inventory-এর চেয়ে কখন ভালো?
19. [উন্নত স্তর] `keyed_groups` দিয়ে tag-ভিত্তিক গ্রুপ তৈরির একটি উদাহরণ দিন।
20. [মধ্যম স্তর] SSH hardening-এর তিনটি অপরিহার্য পদক্ষেপ লিখুন।
21. [মধ্যম স্তর] `lineinfile` module দিয়ে `sshd_config` বদলানো কেন নিরাপদ?
22. [উন্নত স্তর] TechNova-র MERN অ্যাপে rolling update কীভাবে করবেন? ধাপগুলো লিখুন।
23. [উন্নত স্তর] Terraform ও Ansible-এর দায়িত্বের সীমানা কোথায়? একটি উদাহরণ দিন।
24. [মধ্যম স্তর] `community.docker.docker_container` দিয়ে কনটেইনার চালানোর সুবিধা কী?
25. [মধ্যম স্তর] `ansible-lint` CI pipeline-এ রাখলে কী লাভ হয়?
26. [মধ্যম স্তর] Molecule দিয়ে role টেস্ট করার ধাপগুলো সংক্ষেপে লিখুন।
27. [উন্নত স্তর] AWX-এ job template, inventory ও credential কীভাবে একসঙ্গে কাজ করে?
28. [উন্নত স্তর] GitOps পদ্ধতিতে playbook পরিবর্তনের অনুমোদন প্রক্রিয়া কেমন হওয়া উচিত?
29. [মধ্যম স্তর] `serial` ও `max_fail_percentage` একসঙ্গে ব্যবহারের উদ্দেশ্য কী?
30. [উন্নত স্তর] প্রোডাকশন সার্ভারে Ansible চালানোর আগে কোন তিনটি যাচাই অবশ্যই করবেন?

অংশ C — পরিস্থিতিভিত্তিক প্রশ্ন

২০টি ট্রাবলশুটিং পরিস্থিতি

প্রতিটি পরিস্থিতিতে লক্ষণ (symptom) দেওয়া আছে। সম্ভাব্য কারণ ও সমাধান নির্ণয় করুন। সমাধান আছে অংশ F-এ।

1. [শুরুর স্তর] **লক্ষণ:** `ansible all -m ping` চালালে সব হোস্টে `UNREACHABLE! Permission denied (publickey)` আসে। সম্ভাব্য কারণ ও সমাধান কী?
2. [শুরুর স্তর] **লক্ষণ:** Playbook চালালে `ERROR! Syntax Error while loading YAML` দেখায়। সম্ভাব্য কারণ ও সমাধান কী?
3. [মধ্যম স্তর] **লক্ষণ:** `ansible-playbook site.yml` চালালে `FAILED! [Errno 13] Permission denied` আসে, যদিও SSH সংযোগ ঠিক আছে। সম্ভাব্য কারণ ও সমাধান কী?
4. [মধ্যম স্তর] **লক্ষণ:** Playbook দুইবার চালালে দ্বিতীয়বারও সব task `changed` দেখায়, `ok` নয়। সম্ভাব্য কারণ ও সমাধান কী?
5. [মধ্যম স্তর] **লক্ষণ:** nginx কনফিগ বদলালেও সার্ভিস restart হয় না। সম্ভাব্য কারণ ও সমাধান কী?
6. [মধ্যম স্তর] **লক্ষণ:** `ansible-vault decrypt secrets.yml` চালালে `Decryption failed` আসে। সম্ভাব্য কারণ ও সমাধান কী?
7. [মধ্যম স্তর] **লক্ষণ:** `when: ansible_facts['os_family'] == "Debian"` শর্তটি Ubuntu হোস্টেও কাজ করে না। সম্ভাব্য কারণ ও সমাধান কী?
8. [উন্নত স্তর] **লক্ষণ:** `loop` দিয়ে পাঁচটি প্যাকেজ ইনস্টল করলে শুধু প্রথমটি ইনস্টল হয়। সম্ভাব্য কারণ ও সমাধান কী?
9. [মধ্যম স্তর] **লক্ষণ:** Template রেন্ডার করলে `AnsibleUndefinedVariable` ত্রুটি আসে। সম্ভাব্য কারণ ও সমাধান কী?
10. [উন্নত স্তর] **লক্ষণ:** Dynamic inventory (aws_ec2) চালালে কোনো হোস্ট পাওয়া যায় না, যদিও EC2 instance চলছে। সম্ভাব্য কারণ ও সমাধান কী?
11. [মধ্যম স্তর] **লক্ষণ:** `git` module দিয়ে কোড pull করলে `Permission denied (publickey)` আসে। সম্ভাব্য কারণ ও সমাধান কী?
12. [উন্নত স্তর] **লক্ষণ:** Rolling update-এর সময় অর্ধেক সার্ভার নতুন কোডে, অর্ধেক পুরোনো কোডে — ব্যবহারকারীরা অসামঞ্জস্য দেখছেন। সম্ভাব্য কারণ ও সমাধান কী?
13. [মধ্যম স্তর] **লক্ষণ:** `systemd` module দিয়ে Node.js সার্ভিস start করলে `Failed to start` আসে, কিন্তু হাতে `node app.js` চলে। সম্ভাব্য কারণ ও সমাধান কী?
14. [উন্নত স্তর] **লক্ষণ:** Playbook `--check` মোডে সফল, কিন্তু আসল চালনায় ব্যর্থ হয়। সম্ভাব্য কারণ ও সমাধান কী?
15. [মধ্যম স্তর] **লক্ষণ:** `ansible-galaxy install` করা role-টি playbook-এ পাওয়া যায় না (`role not found`)। সম্ভাব্য কারণ ও সমাধান কী?
16. [উন্নত স্তর] **লক্ষণ:** একই playbook control node-এ চলে, কিন্তু AWX job-এ ব্যর্থ হয়। সম্ভাব্য কারণ ও সমাধান কী?
17. [মধ্যম স্তর] **লক্ষণ:** `copy` module দিয়ে ফাইল বসালে owner root হয়ে যায়, অ্যাপ পড়তে পারে না। সম্ভাব্য কারণ ও সমাধান কী?
18. [উন্নত স্তর] **লক্ষণ:** Vault-এনক্রিপ্ট করা variable-সহ playbook CI-তে ব্যর্থ হয়, লোকালে চলে। সম্ভাব্য কারণ ও সমাধান কী?
19. [মধ্যম স্তর] **লক্ষণ:** `ufw` task চালানোর পর SSH সংযোগ বিচ্ছিন্ন হয়ে যায়। সম্ভাব্য কারণ ও সমাধান কী?

20. [উন্নত স্তর] **লক্ষণ:** Playbook ৫০টি হোস্টে খুব ধীরে চলে (৩০ মিনিটের বেশি)। সম্ভাব্য কারণ ও সমাধান কী?

অংশ D — ইনসিডেন্ট কেস স্টাডি

১০টি প্রোডাকশন ইনসিডেন্ট কেস স্টাডি

প্রতিটি কেস স্টাডি পড়ুন এবং শেষের প্রশ্নগুলোর উত্তর দিন। আলোচনার পয়েন্ট আছে অংশ F-এ।

কেস ১ — Vault পাসওয়ার্ড Git-এ ফাঁস (TechNova, ২০২৫)

পটভূমি: TechNova-র জুনিয়র ইঞ্জিনিয়ার রাফি MERN অ্যাপের ডেটাবেস পাসওয়ার্ড Vault-এ এনক্রিপ্ট করে `group_vars/all/vault.yml`-এ রাখেন। তাড়াহুড়োয় তিনি Vault পাসওয়ার্ডটি `vault-pass.txt` নামে একই repository-তে commit করে GitHub-এ push করেন। Repository-টি তখন public ছিল। **ঘটনা:** দুই দিন পর একজন নিরাপত্তা গবেষক repository-টি দেখে পাসওয়ার্ড ফাইলটি খুঁজে পান এবং দায়িত্বশীলভাবে TechNova-কে জানান। ততক্ষণে Vault ফাইলটি public-এ ছিল। TechNova সব secret ঘুরিয়ে (rotate) দেয়, repository private করে এবং পাসওয়ার্ড ফাইলটি Git history থেকে মুছে ফেলে। কোনো ডেটা চুরির প্রমাণ পাওয়া যায়নি, তবে ঝুঁকি ছিল মারাত্মক। **প্রশ্ন:** (ক) কোন কোন ভুল একসঙ্গে এই ঘটনা ঘটিয়েছে? (খ) Vault পাসওয়ার্ড নিরাপদে পরিচালনার সঠিক পদ্ধতি কী? (গ) Git history থেকে secret মুছে ফেলার পরও কেন সব secret rotate করা জরুরি ছিল?

কেস ২ — ভুল inventory-তে প্রোডাকশন মুছে ফেলা (TechNova, ২০২৫)

পটভূমি: TechNova-র staging ও production inventory একই ডিরেক্টরিতে ছিল — `inventory/staging.yml` ও `inventory/prod.yml`। একটি cleanup playbook-এ `hosts: all` লেখা ছিল এবং এটি `/tmp` পরিষ্কার করার কথা ছিল। **ঘটনা:** এক ইঞ্জিনিয়ার ভুল করে `-i inventory/prod.yml`-এর বদলে playbook-টি staging ধরে চালান, কিন্তু playbook-এ `hosts: all` থাকায় এটি production-এর সব সার্ভারের `/var/log` মুছে ফেলে। লগ হারিয়ে যায়, তবে অ্যাপ চলতে থাকে। পুনরুদ্ধার সম্ভব হয়নি। **প্রশ্ন:** (ক) কোন নকশাগত ভুল এই ঘটনাকে সম্ভব করেছে? (খ) production inventory আলাদা রাখার ও `--limit` ব্যবহারের নিয়ম কী হওয়া উচিত? (গ) destructive task-এ কোন সুরক্ষা (guard) যোগ করা যেত?

কেস ৩ — Handler না চলায় পুরোনো কনফিগে চলা nginx (TechNova, ২০২৬)

পটভূমি: TechNova nginx-এর `client_max_body_size` 1M থেকে 10M-এ বাড়াতে playbook চালায়। Template task-টি `changed` দেখায় এবং `notify: Reload nginx` দেওয়া ছিল। **ঘটনা:** কিন্তু handler-এর নামে বানান ভুল ছিল — task-এ `Reload nginx`, handler-এ `Reload Nginx`। Ansible কোনো ত্রুটি না দেখিয়ে handler চালায়নি। ফলে ফাইল বদলালেও nginx পুরোনো কনফিগেই চলতে থাকে এবং বড় ফাইল আপলোড ব্যর্থ হতে থাকে। সমস্যাটি ধরা পড়ে তিন দিন পর। **প্রশ্ন:** (ক) Ansible কেন handler না পেলেও ত্রুটি দেয়নি? (খ) এই ধরনের ভুল ধরার জন্য কোন পরীক্ষা (lint/টেস্ট) ব্যবহার করা যায়? (গ) deployment-এর পর কনফিগ কার্যকর হয়েছে কি না, তা যাচাই করার উপায় কী?

কেস ৪ — Non-idempotent shell task-এ ডেটা দ্বিগুণ (TechNova, ২০২৬)

পটভূমি: একজন ইঞ্জিনিয়ার MongoDB-তে seed ডেটা ঢোকাতে `shell: mongoimport --db app --collection users /tmp/users.json` task লেখেন। **ঘটনা:** Playbook-টি তিনবার চালানো হয় — প্রথমবার সফল, পরের দুইবারও `changed` দেখায় এবং প্রতিবার একই ব্যবহারকারী আবার ঢুকে যায়। ফলে users collection-এ প্রতিটি ব্যবহারকারীর তিনটি করে কপি তৈরি হয়। অ্যাপে লগইন সমস্যা দেখা দেয়। ডেটা হাতে পরিষ্কার করতে হয়। **প্রশ্ন:** (ক) এই task কেন idempotent নয়? (খ) `creates` / `removes` বা শর্ত দিয়ে কীভাবে idempotent করা যেত? (গ) seed ডেটার জন্য কোন module বা পদ্ধতি ভালো হতো?

কেস ৫ — Dynamic inventory-তে ভুল tag-এ ভুল সার্ভারে ডেপ্লয় (TechNova, ২০২৬)

পটভূমি: TechNova AWS-এ `aws_ec2` dynamic inventory ব্যবহার করে। গ্রুপ তৈরি হয় tag `Env`-এর মান দিয়ে — `Env=prod` ও `Env=staging`। **ঘটনা:** এক নতুন EC2 instance-এ ভুল করে `Env=prod` tag বসানো হয়, যদিও সেটি staging-এর জন্য তৈরি হয়েছিল। পরের deployment-এ playbook-টি ওই staging instance-কেও production হিসেবে ধরে নতুন কোড ডেপ্লয় করে এবং production ডেটাবেসের দিকে নির্দেশ করে। Staging ডেটা production ডেটাবেসে লেখা হয়। **প্রশ্ন:** (ক) tag-ভিত্তিক inventory-তে এই ঝুঁকি কেন থাকে? (খ) deployment-এর আগে হোস্ট তালিকা যাচাই করার উপায় কী? (গ) staging ও production-এর নেটওয়ার্ক আলাদা রাখলে কীভাবে ঝুঁকি কমে?

কেস ৬ — AWX-এ credential ভাগাভাগি ও অননুমোদিত job (TechNova, ২০২৬)

পটভূমি: TechNova AWX-এ একটি মাত্র admin অ্যাকাউন্ট সবাই ব্যবহার করত। Job template-গুলোতে production SSH credential সংযুক্ত ছিল। **ঘটনা:** এক ইন্টার্ন ভুল job template চালিয়ে production-এ database restart job চালিয়ে দেন। ডেটাবেস পাঁচ মিনিট বন্ধ থাকে, অ্যাপের ব্যবহারকারীরা ত্রুটি দেখেন। কে job চালিয়েছে তা audit log-এ শুধু "admin" দেখায়, ব্যক্তি শনাক্ত করা যায়নি। **প্রশ্ন:** (ক) ভাগ করা অ্যাকাউন্ট কেন audit-এর জন্য বিপজ্জনক? (খ) AWX-এ role-based access control (RBAC) কীভাবে এই ঘটনা ঠেকাতে পারত? (গ) production job-এ approval ধাপ যোগ করার সুবিধা কী?

কেস ৭ — Terraform ও Ansible-এর দ্বন্দ্বে সার্ভার বারবার তৈরি (TechNova, ২০২৬)

পটভূমি: TechNova Terraform দিয়ে EC2 instance তৈরি করে এবং Ansible দিয়ে তার ভেতরে সফটওয়্যার ইনস্টল করে। একটি playbook-এ `hostname` বদলানোর task ছিল। **ঘটনা:** Ansible hostname বদলানোর পর Terraform পরের `apply`-এ মনে করে instance-টি বদলে গেছে এবং সেটি ধ্বংস করে নতুন instance তৈরি করে। এতে অ্যাপের ডেটা হারিয়ে যায়, কারণ ডেটা instance-এর local ডিস্কে ছিল। ঘটনাটি দুইবার ঘটে, তারপর কারণ ধরা পড়ে। **প্রশ্ন:** (ক) Terraform ও Ansible-এর দায়িত্বের সীমানা এখানে কীভাবে ভাঙা হয়েছে? (খ) Terraform-এর `lifecycle { ignore_changes }` কখন ব্যবহার করা যায়? (গ) ডেটা কেন instance-এর local ডিস্কে রাখা উচিত নয়?

কেস ৮ — Rolling update-এ health check ছাড়া বিপর্যয় (TechNova, ২০২৬)

পটভূমি: TechNova-র চারটি web সার্ভারে `serial: 1` দিয়ে rolling update চালানো হয়, কিন্তু কোনো health check task ছিল না। **ঘটনা:** নতুন কোডে একটি bug ছিল — অ্যাপ চালু হলেও `/health` endpoint 500 দিচ্ছিল। Ansible সার্ভিস চালু দেখে পরের সার্ভারে এগিয়ে যায়। চারটি সার্ভারেই ভাঙা কোড ডেপ্লয় হয় এবং সাইট পুরোপুরি বন্ধ হয়ে যায়। Rollback করতে ৪০ মিনিট লাগে। **প্রশ্ন:** (ক) health check না থাকায় rolling update কেন বিপজ্জনক? (খ) `uri` module দিয়ে health check কীভাবে যোগ করা যায়? (গ) `max_fail_percentage` ও automatic rollback কীভাবে সাহায্য করত?

কেস ৯ — `shell` module-এ injection ও ভুল কমান্ড (TechNova, ২০২৫)

পটভূমি: একটি playbook-এ ব্যবহারকারীর দেওয়া অ্যাপের নাম সরাসরি shell কমান্ডে বসানো হতো: `shell: systemctl restart {{ app_name }}`। **ঘটনা:** একবার `app_name` variable-এ ভুল করে `nginx; rm -rf /var/log/*` মান ঢুকে যায়। Shell কমান্ডটি দুটি অংশে চলে এবং সব লগ মুছে যায়। ভাগ্যক্রমে ডেটা অক্ষত ছিল। **প্রশ্ন:** (ক) `shell` module-

এ variable বসানো কেন বিপজ্জনক? (খ) `systemd` module ব্যবহার করলে এই ঝুঁকি কীভাবে কমে? (গ) variable যাচাই (validation) করার উপায় কী?

কেস ১০ — Molecule ছাড়া role-এর ভাঙা পরিবর্তন production-এ (TechNova, ২০২৬)

পটভূমি: TechNova-র nginx role-এ একজন ইঞ্জিনিয়ার template-এ একটি লাইন বদলান — কিন্তু কোনো টেস্ট চালাননি, কারণ Molecule সেটআপ ছিল না। **ঘটনা:** বদলানো template-এ Jinja2 সিনট্যাক্স ভুল ছিল। Role-টি production playbook-এ চলার সময় ব্যর্থ হয় — কিন্তু ততক্ষণে অর্ধেক সার্ভারে পুরোনো কনফিগ মুছে নতুন (ভাঙা) কনফিগ বসে গেছে, আর বাকি অর্ধেকে পুরোনো কনফিগ রয়ে গেছে। দুই ধরনের সার্ভার মিলে অসামঞ্জস্য তৈরি হয়। **প্রশ্ন:** (ক) কোন টেস্ট এই ভুল আগেই ধরতে পারত? (খ) CI-তে `ansible-lint` ও Molecule যোগ করলে কী লাভ হয়? (গ) template বদলের আগে `--check --diff` চালালে কী দেখা যেত?

অংশ E — চূড়ান্ত পরীক্ষা

চূড়ান্ত পরীক্ষা — Ansible Automation (১০০ নম্বর, ৩ ঘণ্টা)

এই পরীক্ষার দুটি অংশ আছে — তত্ত্ব (অংশ ১) ও ব্যবহারিক (অংশ ২)। মোট ১০০ নম্বর। পাস নম্বর ৬০। ল্যাব পরিবেশ: Ubuntu 24.04 control node, Ansible Core 2.17+, TechNova ল্যাব হোস্ট (`203.0.113.10` web, `203.0.113.20` db, `203.0.113.30` app)। সব নমুনা আউটপুট চিত্রণমূলক।

অংশ ১ — তত্ত্ব (২০টি প্রশ্ন, ৪০ নম্বর, ৬০ মিনিট)

প্রতিটি প্রশ্নের উত্তর সংক্ষেপে লিখুন (প্রতিটি ২ নম্বর)।

1. Agentless automation-এর সুবিধা ও সীমাবদ্ধতা লিখুন।
2. Inventory-তে `ansible_host`, `ansible_user` ও `ansible_port`-এর কাজ লিখুন।
3. Ad-hoc কমান্ড কখন যথেষ্ট এবং কখন playbook দরকার — উদাহরণসহ লিখুন।
4. YAML-এ indentation ও list লেখার নিয়ম লিখুন।
5. `command` ও `shell` module-এর পার্থক্য ও নিরাপত্তা ঝুঁকি লিখুন।
6. Variable precedence-এর পাঁচটি স্তর ক্রমানুসারে লিখুন।
7. Fact কী এবং `when` শর্তে fact ব্যবহারের একটি উদাহরণ দিন।
8. `register` দিয়ে task-এর ফল ধরে পরের task-এ ব্যবহারের উদাহরণ দিন।
9. Jinja2-এ `{{ }}`, `{% %}` ও `{# #}`-এর কাজ লিখুন।
10. Handler কেন শুধু changed task-এ চলে এবং কেন play-এর শেষে চলে?
11. Idempotency-এর সংজ্ঞা ও যাচাই করার পদ্ধতি লিখুন।
12. Role-এর ডিরেক্টরি কাঠামো আঁকুন ও প্রতিটি ডিরেক্টরির কাজ লিখুন।
13. Collection ও role-এর পার্থক্য লিখুন।
14. Vault দিয়ে secret পরিচালনার নিরাপদ পদ্ধতি লিখুন।
15. `become`, `become_user` ও `become_method`-এর কাজ লিখুন।
16. Dynamic inventory-এর সুবিধা ও একটি ক্লাউড উদাহরণ দিন।
17. SSH hardening-এর চারটি পদক্ষেপ লিখুন।
18. Terraform ও Ansible-এর দায়িত্বের সীমানা লিখুন।
19. Molecule দিয়ে role টেস্ট করার ধাপগুলো লিখুন।
20. AWX-এ job template, inventory, credential ও schedule কীভাবে একসঙ্গে কাজ করে লিখুন।

অংশ ২ — ব্যবহারিক: TechNova MERN স্ট্যাক ডেপ্লয় (৬০ নম্বর, ২ ঘণ্টা)

ল্যাবে TechNova-র MERN স্ট্যাক (MongoDB, Express, Node.js API, React/Next.js frontend, nginx) Ansible দিয়ে ডেপ্লয় করুন। সব কাজ `~/technova-ansible/` ডিরেক্টরিতে করুন।

কাজ	বিবরণ	নম্বর	সময়
		৮	

কাজ ১	Inventory তৈরি: `web` (203.0.113.10), `app` (203.0.113.30), `db` (203.0.113.20) গ্রুপ, YAML ফরম্যাটে, `ansible-inventory --graph` দিয়ে যাচাই		১৫ মিনিট
কাজ ২	Playbook লিখুন যা সব হোস্টে common hardening করে: ufw (SSH/HTTP/HTTPS), fail2ban ইনস্টল, root login নিষ্ক্রিয়, unattended-upgrades চালু	১২	২৫ মিনিট
কাজ ৩	MongoDB role তৈরি: db হোস্টে MongoDB ইনস্টল, সার্ভিস চালু, Vault-এনক্রিপ্ট করা admin পাসওয়ার্ড দিয়ে ব্যবহারকারী তৈরি	১২	২৫ মিনিট
কাজ ৪	Node.js API ডেপ্লয়: app হোস্টে Node.js 20 ইনস্টল, `git` module-এ কোড checkout, `npm ci`, systemd service, `.env` template (Vault variable থেকে)	১৪	৩০ মিনিট
কাজ ৫	nginx রিভার্স প্রক্সি: web হোস্টে Jinja2 template দিয়ে virtual host, handler দিয়ে reload, `uri` module দিয়ে `/health` যাচাই	৮	১৫ মিনিট
কাজ ৬	Idempotency প্রমাণ: পুরো playbook দ্বিতীয়বার চালিয়ে সব task `ok` দেখান (কোনো `changed` নয়) এবং `--check --diff` আউটপুট সংরক্ষণ করুন	৬	১০ মিনিট

পরীক্ষার নিয়ম: কোনো আসল secret ব্যবহার করবেন না — সব পাসওয়ার্ড `CHANGE-ME-...` প্লেসহোল্ডার দিয়ে Vault-এ রাখুন। প্রতিটি কাজের আউটপুট `~/technova-ansible/exam-output/`-এ সংরক্ষণ করুন। সময় শেষ হলে কাজ থামান — অসম্পূর্ণ কাজের জন্যও আংশিক নম্বর আছে।

অংশ F — উত্তর নির্দেশিকা (১ম অংশ)

উত্তর নির্দেশিকা — MCQ ১–৬০ ও সংক্ষিপ্ত প্রশ্ন

প্রতিটি MCQ-এর সঠিক বিকল্প ও সংক্ষিপ্ত ব্যাখ্যা দেওয়া আছে। আগে নিজে চেষ্টা করে তারপর মিলিয়ে নিন।

MCQ উত্তর ১–৬০

প্রশ্ন	সঠিক উত্তর	ব্যাখ্যা
১	ক	Configuration Management-এর উদ্দেশ্য হলো সার্ভারের অবস্থা নির্ধারিত, পুনরাবৃত্তিযোগ্য ও সংস্করণ-নিয়ন্ত্রিত রাখা।
২	ক	Ansible agentless — managed node-এ SSH ও Python থাকলেই চলে, আলাদা agent ইনস্টল করতে হয় না।
৩	ক	Control node হলো সেই মেশিন যেখান থেকে Ansible কমান্ড ও playbook চালানো হয়।
৪	খ	Ansible মূলত Python-এ লেখা।
৫	খ	Managed node-এ শুধু Python ও SSH দরকার হয়, তাই কোনো agent ইনস্টল ও পরিচালনা করতে হয় না।
৬	ক	Inventory হলো managed host-গুলোর তালিকা ও গ্রুপের সংজ্ঞা।
৭	খ	INI inventory-তে গ্রুপ লেখা হয় `[webservers]` আকারে।
৮	খ	`ansible all --list-hosts` inventory-র সব হোস্টের তালিকা দেখায়, সংযোগ করে না।
৯	খ	গ্রুপ ভাগ করলে একই playbook-এর ভিন্ন play ভিন্ন গ্রুপে ভিন্ন task চালাতে পারে।
১০	ক	Ubuntu-তে `apt` বা `pip` দিয়ে অফিসিয়াল পদ্ধতিতে Ansible ইনস্টল করা যায়।
১১	ক	`ansible.cfg` Ansible-এর ডিফল্ট সেটিং নির্ধারণ করে।
১২	খ	Host key checking বন্ধ থাকলে ভুয়া হোস্টের সঙ্গে সংযোগের ঝুঁকি (man-in-the-middle) বাড়ে।
১৩	খ	YAML inventory-তে `all: hosts: web1: ansible_host: ...` কাঠামো সঠিক।
১৪	ক	`ansible_host` হোস্টের আসল সংযোগ ঠিকানা নির্ধারণ করে, যা inventory নাম থেকে আলাদা হতে পারে।
১৫	খ	একটি হোস্ট একাধিক গ্রুপে থাকতে পারে; variable-এর দ্বন্দ্ব precedence নিয়মে মেটে।
১৬	ক	`children` দিয়ে গ্রুপের ভেতরে গ্রুপ রেখে meta-গ্রুপ তৈরি করা যায়।
১৭	ক	`ansible-inventory --graph` গ্রুপ ও হোস্টের গাছ-আকৃতির কাঠামো দেখায়।
১৮	ক	SSH key ব্যবহারে পাসওয়ার্ড টাইপ করতে হয় না এবং স্বয়ংক্রিয় কাজ নিরাপদে করা যায়।
১৯	খ	SSH ডিফল্ট ২২ পোর্টের বদলে অন্য পোর্টে চললে `ansible_port` বদলাতে হয়।
২০	ক	Configuration drift হলো হাতে বদলের ফলে সার্ভারের অবস্থা নির্ধারিত অবস্থা থেকে সরে যাওয়া।

২১	ক	Ad-hoc কমান্ড একবারের ছোট কাজের জন্য, যেমন ping বা প্যাকেজ চেক।
২২	ক	`ansible all -m ping` ping module দিয়ে connectivity পরীক্ষা করে।
২৩	ক	এই কমান্ড webservers গ্রুপের সব হোস্টে `uptime` চালায়।
২৪	খ	`shell` পাইপ ও রিডাইরেকশন বোঝে; `command` সরাসরি কমান্ড চালায়, shell feature বোঝে না।
২৫	ক	Playbook হলো YAML ফাইলে লেখা Ansible কাজের ধাপের সেট।
২৬	ক	Playbook চালাতে `ansible-playbook site.yml` ব্যবহার করা হয়।
২৭	খ	YAML-এ space দিয়ে ধারাবাহিক indentation দিতে হয়; tab নিষিদ্ধ।
২৮	ক	Play-র `hosts:` নির্ধারণ করে কোন হোস্ট বা গ্রুপে play চলবে।
২৯	ক	`name:` দিলে আউটপুটে ধাপটি পড়া যায় ও ডিবাগ সহজ হয়।
৩০	ক	`--check` dry run — আসলে কিছু না বদলে কী বদলাতো তা দেখায়।
৩১	ক	`--limit` playbook শুধু নির্দিষ্ট হোস্ট বা গ্রুপে চালায়।
৩২	ক	`--tags` দিয়ে শুধু নির্দিষ্ট tag-যুক্ত task চালানো বা বাদ দেওয়া যায়।
৩৩	খ	`gather_facts: false` দিলে fact সংগ্রহের ধাপ বাদ যায়, play দ্রুত শুরু হয়।
৩৪	ক	YAML-এ list লেখা হয় ড্যাশ দিয়ে: `- item1`।
৩৫	ক	`--syntax-check` playbook না চালিয়ে YAML ও সিনট্যাক্স যাচাই করে।
৩৬	ক	Play-তে `become: true` দিলে সব task root (sudo) হিসেবে চলে।
৩৭	খ	এক playbook-এ একাধিক play ক্রমানুসারে চলে, প্রতিটি ভিন্ন হোস্টে বা ভিন্ন কাজে।
৩৮	ক	`--start-at-task` ব্যর্থ playbook-এর মাঝখান থেকে আবার চালাতে কাজে লাগে।
৩৯	ক	Ad-hoc `apt` কমান্ডটি nginx প্যাকেজ ইনস্টল করে।
৪০	ক	`serial: 2` দিলে একসঙ্গে ২টি হোস্টে ব্যাচে play চলে, rolling update সম্ভব হয়।
৪১	ক	Variable playbook-এ পুনঃব্যবহারযোগ্য মান সংরক্ষণ করে।
৪২	ক	Playbook-এ variable লেখা হয় `vars: app_port: 3000` আকারে।
৪৩	ক	`group_vars/webservers.yml` webservers গ্রুপের সব হোস্টের variable নির্ধারণ করে।
৪৪	ক	`host_vars/web1.yml` শুধু web1 হোস্টের নির্দিষ্ট variable দেয়।
৪৫	খ	Precedence-এ সবচেয়ে শক্তিশালী হলো কমান্ড লাইনের `-e` extra vars।
৪৬	খ	Role vars-এর precedence role defaults-এর চেয়ে বেশি।
৪৭	ক	Fact হলো Ansible-এর সংগ্রহ করা হোস্টের তথ্য — OS, IP, মেমরি ইত্যাদি।
৪৮	ক	`ansible_facts['os_family']` হোস্টের OS পরিবার দেয়, যেমন Debian বা RedHat।

৪৯	ক	`ansible all -m setup` সব হোস্টের fact সংগ্রহ করে দেখায়।
৫০	ক	এই `when` শর্তটি শুধু Debian পরিবারের হোস্টে task চালায়।
৫১	ক	একাধিক `when` শর্তে সবগুলো সত্য হলেই task চলে (AND যুক্তি)।
৫২	ক	`register` task-এর আউটপুট একটি variable-এ সংরক্ষণ করে।
৫৩	ক	আগের task কিছু বদলালে `result.changed` সত্য হয়।
৫৪	ক	`default` filter variable না থাকলে বিকল্প মান দেয়।
৫৫	ক	`hostvars` দিয়ে অন্য হোস্টের variable পড়া যায়।
৫৬	ক	`vars_prompt` playbook চলার সময় ব্যবহারকারীর কাছ থেকে মান চায়।
৫৭	খ	একই variable-এর মান group_vars ও host_vars-এ ভিন্ন হলে host_vars জয়ী হয়।
৫৮	ক	`ansible_facts['default_ipv4']['address']` হোস্টের প্রধান IPv4 ঠিকানা দেয়।
৫৯	ক	`set_fact` play চলাকালীন নতুন fact/variable তৈরি করে।
৬০	খ	db গ্রুপে play চালানো বা `when` শর্ত দিয়ে শুধু db হোস্টে MongoDB ইনস্টল করা যায়।

সংক্ষিপ্ত প্রশ্নের মডেল উত্তর

1. **Agentless:** Managed node-এ কোনো agent ইনস্টল করতে হয় না; Ansible SSH (Linux) বা WinRM (Windows) দিয়ে সংযোগ করে এবং Python-এর মাধ্যমে কাজ করে। এতে agent পরিচালনার ঝরঝরে কাজ কমে, তবে প্রতিটি task-এ SSH সংযোগের খরচ থাকে।
2. **গ্রুপের সুবিধা:** (১) একই ধরনের সার্ভারে একসঙ্গে play চালানো যায়; (২) `group_vars` দিয়ে গ্রুপ-ভিত্তিক variable দেওয়া যায়, ফলে কনফিগারেশন সুসংগঠিত থাকে।
3. **Ad-hoc বনাম playbook:** Ad-hoc একবারের ছোট কাজের জন্য (যেমন ping, প্যাকেজ চেক); playbook পুনরাবৃত্তিযোগ্য, সংস্করণ-নিয়ন্ত্রিত ও জটিল কাজের ধাপের সেট, যা বারবার চালানো যায়।
4. **Indentation ভুল:** YAML পার্সার ত্রুটি দেয় (`Syntax Error while loading YAML`) এবং playbook চালানো যায় না; tab ব্যবহার করলেও একই ত্রুটি হয়।
5. **shell কখন:** পাইপ (`|`), রিডাইরেকশন (`>`), shell variable বা glob দরকার হলে `shell` ব্যবহার করুন; নইলে নিরাপদ ও idempotent `command` বা নির্দিষ্ট module ব্যবহার করুন।
6. **--check-এর সীমা:** `--check` মোডে `command` / `shell` task আসলে কী করতো তা অনুমান করতে পারে না; কিছু module check মোড সমর্থন করে না, তাই ফল সবসময় নির্ভরযোগ্য নয়।
7. **Precedence কেন জরুরি:** একই variable একাধিক জায়গায় থাকলে কোন মানটি কার্যকর হবে তা না জানলে ভুল কনফিগারেশন ডেপ্লয় হয়। যেমন `group_vars`-এর `app_port: 3000`-কে `-e app_port=4000` দিয়ে ওভাররাইড করা যায়।
8. **gather_facts: false:** `ansible_facts` পাওয়া যাবে না, তাই OS-ভিত্তিক `when` শর্ত ও fact-নির্ভর template কাজ করবে না; তবে play দ্রুত শুরু হয়।
9. **register + when:** `register: result` task-এর ফল সংরক্ষণ করে; পরের task-এ `when: result.changed` বা `when: result.rc == 0` দিয়ে শর্ত দেওয়া যায়।

10. **Filter উদাহরণ:** `upper` — লেখাকে বড় হাতের করে; `default(3000)` — variable না থাকলে বিকল্প মান দেয়।
11. **copy বনাম template:** ফাইলে কোনো variable রেন্ডার দরকার না হলে `copy` ভালো — সহজ ও দ্রুত; variable থাকলে `template` দরকার।
12. **Handler শেষে কেন:** Handler play-এর শেষে একবার চলে, ফলে একাধিক task একই সার্ভিস বদলালেও restart একবারই হয় — এতে অপ্রয়োজনীয় restart কমে।
13. **Idempotency যাচাই:** Playbook দুইবার চালান — দ্বিতীয়বার সব task `ok` দেখালে (কোনো `changed` নয়) তা idempotent; `--check --diff` দিয়েও যাচাই করা যায়।
14. **defaults বনাম vars:** `defaults/`-এর মান সহজে ওভাররাইড করা যায় (সবচেয়ে কম precedence); `vars/`-এর মানের precedence বেশি, তাই role-এর ভেতরের স্থির মান এখানে রাখা হয়।
15. **Galaxy role ইনস্টল:** `ansible-galaxy role install geerlingguy.nginx` চালান; role `~/.ansible/roles/`-এ ইনস্টল হয়; `requirements.yml` দিয়ে সংস্করণ পিন করা যায়।
16. **Vault পাসওয়ার্ড হারালে:** এনক্রিপ্ট করা ফাইল আর খোলা যাবে না — secret হারিয়ে যাবে। তাই পাসওয়ার্ড নিরাপদ password manager-এ রাখুন এবং secret-গুলোর ব্যাকআপ নিরাপদ জায়গায় রাখুন।
17. **become বনাম become_user:** `become: true` privilege বাড়ায় (ডিফল্ট root); `become_user` নির্ধারণ করে কোন ব্যবহারকারী হিসেবে চলবে, যেমন `postgres`।
18. **Dynamic inventory কখন ভালো:** ক্লাউডে হোস্ট সংখ্যা ঘন ঘন বদলালে — নতুন instance তৈরি হলে হাতে inventory আপডেট করতে হয় না, plugin স্বয়ংক্রিয়ভাবে তালিকা আনে।
19. **keyed_groups উদাহরণ:** AWS tag `Env` দিয়ে `keyed_groups: - key: tags.Env prefix: env` লিখলে `env_prod`, `env_staging` গ্রুপ স্বয়ংক্রিয়ভাবে তৈরি হয়।
20. **SSH hardening:** (১) root login নিষ্ক্রিয় করা; (২) password authentication বন্ধ করে key-only করা; (৩) SSH পোর্ট বদলানো বা শুধু নির্দিষ্ট IP থেকে অনুমতি দেওয়া।
21. **lineinfile কেন নিরাপদ:** এটি নির্দিষ্ট লাইন খুঁজে বদলায়, না থাকলে যোগ করে; বারবার চালালেও একই ফল দেয় — idempotent; পুরো ফাইল ওভাররাইট করার ঝুঁকি নেই।
22. **Rolling update:** `serial: 1` দিয়ে একবারে একটি সার্ভার বদলান, নতুন কোড ডেপ্লয় করুন, `uri` module দিয়ে `/health` যাচাই করুন, সফল হলে পরের সার্ভারে যান।
23. **Terraform বনাম Ansible:** Terraform infrastructure তৈরি করে (VM, নেটওয়ার্ক); Ansible তার ভেতরে সফটওয়্যার কনফিগার করে। যেমন Terraform EC2 তৈরি করে, Ansible তাতে Node.js ইনস্টল করে।
24. **docker_container সুবিধা:** Idempotent উপায়ে কনটেইনার তৈরি, চালু ও বন্ধ করা যায়; `shell: docker run`-এর মতো প্রতিবার নতুন কনটেইনার তৈরি হয় না।
25. **ansible-lint CI-তে:** প্রতিটি commit-এ playbook-এর best practice ও ত্রুটি ধরা পড়ে, ফলে ভাঙা বা অনিরাপদ playbook production-এ যায় না।
26. **Molecule ধাপ:** `molecule init` → `molecule create` (test instance) → `molecule converge` (role চালানো) → `molecule verify` (পরীক্ষা) → `molecule destroy`।
27. **AWX একসঙ্গে:** Job template playbook, inventory ও credential একত্রে বাঁধে; schedule দিলে নির্দিষ্ট সময়ে job চলে; প্রতিটি run-এর log ও audit trail থাকে।
28. **GitOps অনুমোদন:** Playbook পরিবর্তন pull request-এ আসে, review ও CI (lint + Molecule) পাস করলে merge হয়; তারপর CD pipeline তা প্রয়োগ করে — সরাসরি production-এ হাতে চালানো হয় না।

29. **serial + max_fail_percentage:** `serial` ব্যাচ আকার নির্ধারণ করে, `max_fail_percentage` নির্ধারণ করে কত শতাংশ হোস্ট ব্যর্থ হলে পুরো play থামবে — এতে ভাঙা deployment সব সার্ভারে ছড়ায় না।
30. **Production-এর আগে তিন যাচাই:** (১) `--check --diff` দিয়ে কী বদলাবে দেখা; (২) inventory ও `--limit` ঠিক আছে কি না যাচাই; (৩) Vault secret ও ব্যাকআপ/rollback পরিকল্পনা নিশ্চিত করা।

অংশ F — উত্তর নির্দেশিকা (২য় অংশ)

উত্তর নির্দেশিকা — MCQ ৬১–১২০, পরিস্থিতি, কেস স্টাডি ও অ্যাসাইনমেন্ট

MCQ উত্তর ৬১–১২০

প্রশ্ন	সঠিক উত্তর	ব্যাখ্যা
৬১	ক	Loop একই task ভিন্ন ভিন্ন মান দিয়ে বারবার চালায়।
৬২	ক	`loop: [80, 443]` পোর্ট ৮০ ও ৪৪৩ — দুটি মান দিয়ে একই task চালায়।
৬৩	ক	Loop-এ বর্তমান মানটি `item` variable-এ থাকে।
৬৪	ক	`loop_var` দিয়ে loop variable-এর নাম বদলানো যায়, nested loop-এ দ্বন্দ্ব এড়ানো যায়।
৬৫	ক	Jinja2-এ variable লেখা হয় `{{ variable }}` আকারে।
৬৬	ক	Jinja2-এ শর্ত লেখা হয় `{% if ... %} ... {% endif %}` আকারে।
৬৭	ক	`int` filter মানটিকে পূর্ণসংখ্যায় রূপান্তর করে।
৬৮	ক	`upper` filter `"hello"`-কে `HELLO` করে।
৬৯	ক	`template` module Jinja2 template রেন্ডার করে remote হোস্টে ফাইল হিসেবে বসায়।
৭০	খ	`copy` ফাইল হুবহু কপি করে; `template` variable রেন্ডার করে বসায়।
৭১	ক	Handler হলো শুধু notify পেলে চলা বিশেষ task, যেমন সার্ভিস restart।
৭২	খ	Notify করা task changed হলে handler play-এর শেষে চলে।
৭৩	ক	Idempotency মানে একই playbook বারবার চালালে প্রথমবারের পর আর কোনো বদল হয় না।
৭৪	খ	`shell: echo "x" >> /tmp/file` প্রতিবার চালালে লাইন যোগ হয়, তাই idempotent নয়।
৭৫	খ	Handler একাধিকবার notify পেলেও play-এর শেষে মাত্র একবার চলে।
৭৬	ক	Role হলো playbook-এর পুনঃব্যবহারযোগ্য, সুসংগঠিত অংশ।
৭৭	ক	Role-এর tasks থাকে `roles/<name>/tasks/main.yml`-এ।
৭৮	ক	`ansible-galaxy init myrole` role-এর আদর্শ ডিরেক্টরি কাঠামো তৈরি করে।
৭৯	ক	Ansible Galaxy হলো community role ও collection খুঁজে পাওয়া ও ইনস্টল করার হাব।
৮০	খ	Collection হলো role, module, plugin ও playbook-এর বড় বিতরণ একক; role তার ভেতরের একটি অংশ হতে পারে।
৮১	ক	Ansible Vault পাসওয়ার্ড ও secret এনক্রিপ্ট করে সংরক্ষণ করে।

৮২	ক	Vault ফাইল তৈরি করতে `ansible-vault create secrets.yml` ব্যবহার করা হয়।
৮৩	ক	Vault-এনক্রিপ্ট করা ফাইল ব্যবহার করতে `--ask-vault-pass` বা `--vault-password-file` দরকার।
৮৪	ক	`encrypt_string` একটি string এনক্রিপ্ট করে variable হিসেবে ব্যবহারের উপযোগী আউটপুট দেয়।
৮৫	খ	Vault পাসওয়ার্ড ফাঁস হলে সব secret উন্মুক্ত হয়ে যায়, তাই Git-এ commit করা বিপজ্জনক।
৮৬	ক	`become_user: postgres` দিলে task postgres ব্যবহারকারী হিসেবে চলে।
৮৭	ক	`become_method: su` sudo-র বদলে su দিয়ে privilege বাড়াতে ব্যবহার করা হয়।
৮৮	খ	`ansible_become_password` Vault-এনক্রিপ্ট করা variable-এ রাখা উচিত।
৮৯	ক	Dynamic inventory ক্লাউড বা CMDB থেকে স্বয়ংক্রিয়ভাবে হোস্ট তালিকা তৈরি করে।
৯০	ক	AWS EC2 dynamic inventory-তে `aws_ec2` inventory plugin ব্যবহার করা হয়।
৯১	ক	`keyed_groups` হোস্টের tag বা বৈশিষ্ট্য অনুযায়ী স্বয়ংক্রিয়ভাবে গ্রুপ তৈরি করে।
৯২	ক	এই কমান্ড dynamic inventory থেকে পাওয়া হোস্ট ও গ্রুপের গাছ দেখায়।
৯৩	ক	Hardening হলো সার্ভারের আক্রমণ-ক্ষেত্র কমিয়ে নিরাপত্তা শক্ত করা।
৯৪	ক	SSH hardening-এর প্রথম পদক্ষেপ root login নিষ্ক্রিয় করা ও key-only authentication চালু করা।
৯৫	ক	UFW পরিচালনায় `ufw` module ব্যবহার করা হয়।
৯৬	ক	`lineinfile` নির্দিষ্ট লাইন খুঁজে বদলায় ও বারবার চালালেও একই ফল দেয় — idempotent।
৯৭	ক	Fail2ban বারবার ভুল লগইন চেষ্টা করা IP-কে সাময়িকভাবে ব্লক করে।
৯৮	খ	CIS benchmark-এর নিয়মগুলো community role বা নিজের role দিয়ে কোড হিসেবে প্রয়োগ করা ভালো।
৯৯	ক	`authorized_key` module ব্যবহারকারীর `authorized_keys`-এ SSH public key যোগ করে।
১০০	ক	Unattended-upgrades চালু থাকলে নিরাপত্তা প্যাচ স্বয়ংক্রিয়ভাবে ইনস্টল হয়।
১০১	খ	MERN ডেপ্লয়ের সঠিক ক্রম: OS প্রস্তুতি → Node.js/MongoDB ইনস্টল → কোড ডেপ্লয় → সার্ভিস চালু → nginx রিভার্স প্রক্সি।
১০২	ক	systemd service হলে সার্ভার রিস্টার্টে অ্যাপ নিজে থেকে চালু হয় ও লগ পরিচালনা সহজ হয়।
১০৩	ক	`git` module দিয়ে নির্দিষ্ট branch/commit checkout করা যায় ও idempotent উপায়ে আপডেট করা যায়।
১০৪	খ	Zero-downtime deployment-এ `serial` দিয়ে rolling update ও health check করে ধাপে ধাপে সার্ভার বদলানো হয়।
১০৫	খ	MongoDB পাসওয়ার্ড Vault-এনক্রিপ্ট করা variable থেকে template-এ রেন্ডার করা উচিত।
১০৬	ক	Cloud provisioning হলো ক্লাউডে VM, নেটওয়ার্ক ও স্টোরেজের মতো সম্পদ কোড দিয়ে তৈরি করা।
১০৭	ক	AWS EC2 instance তৈরি করতে `amazon.aws.ec2_instance` module ব্যবহার করা হয়।

১০৮	ক	Terraform infrastructure তৈরি করে, Ansible তার ভেতরে সফটওয়্যার কনফিগার করে — দুটি ভিন্ন স্তরের কাজ।
১০৯	ক	Terraform output বা `terraform-inventory` থেকে dynamic inventory তৈরি করা যায়।
১১০	ক	Docker কনটেইনার পরিচালনায় `community.docker.docker_container` module ব্যবহার করা হয়।
১১১	খ	Kubernetes-এ Ansible `kubernetes.core` collection দিয়ে manifest প্রয়োগ ও resource পরিচালনা করে।
১১২	ক	`ansible-lint` playbook ও role-এর best practice ও সিনট্যাক্স ত্রুটি পরীক্ষা করে।
১১৩	ক	Molecule role টেস্ট করার জন্য — container/VM-এ role চালিয়ে যাচাই করা হয়।
১১৪	ক	`--check --diff` দিলে কী বদলাতো তা diff আকারে দেখা যায়, আসলে কিছু না বদলে।
১১৫	ক	AWX হলো Ansible-এর ওয়েব UI, REST API ও scheduling-সহ open-source অটোমেশন প্ল্যাটফর্ম।
১১৬	ক	AWX job template playbook, inventory ও credential একত্রে বেঁধে পুনরায় চালানোর উপযোগী সংজ্ঞা দেয়।
১১৭	ক	AWX credential SSH key, Vault পাসওয়ার্ড ও ক্লাউড key নিরাপদে সংরক্ষণ ও job-এ ব্যবহার করে।
১১৮	খ	GitOps-এ playbook Git repository-তে থাকে, যেখানে পরিবর্তন review ও audit হয় এবং CI/CD তা প্রয়োগ করে।
১১৯	ক	Playbook-এ `environment:` task চলার সময়ের environment variable নির্ধারণ করে, যেমন proxy সেটিং।
১২০	ক	প্রোডাকশনে সবচেয়ে গুরুত্বপূর্ণ অভ্যাস: secret Vault-এ রাখা, least privilege, --check দিয়ে যাচাই ও audit trail রাখা।

পরিস্থিতির সমাধান

1. **SSH key সমস্যা:** Control node-এর public key target হোস্টের `authorized_keys`-এ নেই, বা `ansible_user` ভুল। `ssh-copy-id` দিয়ে key বসান, `ansible_user` যাচাই করুন এবং `ssh -i key user@host` দিয়ে হাতে পরীক্ষা করুন।
2. **YAML সিনট্যাক্স:** Indentation ভুল, tab ব্যবহার, বা কোলনের পর space নেই। `ansible-playbook --syntax-check` ও YAML linter দিয়ে লাইনটি খুঁজে ঠিক করুন।
3. **Permission denied:** SSH সংযোগ ঠিক আছে কিন্তু task-টি root ক্ষমতা চায়। Play বা task-এ `become: true` যোগ করুন এবং sudo অধিকার যাচাই করুন।
4. **Changed প্রতিবার:** Task idempotent নয় — সম্ভবত `shell` / `command` ব্যবহার করা হয়েছে। নির্দিষ্ট module (যেমন `apt`, `copy`, `lineinfile`) ব্যবহার করুন বা `creates` শর্ত যোগ করুন।
5. **Handler চলেনি:** Handler-এর নামে বানান ভুল, বা notify করা task `ok` (changed নয়) ছিল। নাম মিলিয়ে নিন এবং `--check` মোডে handler চলে না — এটিও মনে রাখুন।
6. **Vault decryption ব্যর্থ:** ভুল Vault পাসওয়ার্ড দেওয়া হয়েছে বা ভিন্ন Vault ID ব্যবহার করা হয়েছে। সঠিক পাসওয়ার্ড ফাইল বা `--ask-vault-pass` দিয়ে আবার চেষ্টা করুন; পাসওয়ার্ড হারিয়ে গেলে ফাইলটি আর খোলা যাবে না।

7. **Fact শর্ত কাজ করে না:** `gather_facts: false` থাকলে fact পাওয়া যায় না, বা `ansible_facts['os_family']`-এর বদলে পুরোনো `ansible_os_family` ব্যবহার করা হয়েছে। Fact সংগ্রহ চালু করুন ও সঠিক নাম ব্যবহার করুন।
8. **Loop-এ শুধু প্রথমটি:** `loop`-এর বদলে `with_items` ও `item`-এর ভুল ব্যবহার, বা task-এ `name={{ item }}` লেখায় প্রথম মান আটকে গেছে। `loop` ও `item` সঠিকভাবে ব্যবহার করুন।
9. **Undefined variable:** Variable-টি কোনো স্তরেই নির্ধারিত হয়নি, বা নামে বানান ভুল। `ansible-inventory --host web1` দিয়ে variable দেখুন, `default` filter যোগ করুন।
10. **Dynamic inventory খালি:** AWS credential ভুল, region ভুল, বা `filters`-এ কোনো instance মেলে না। `AWS_ACCESS_KEY_ID` ও region যাচাই করুন এবং `ansible-inventory -i inventory.aws_ec2.yml --list` দিয়ে দেখুন।
11. **Git publickey ব্যর্থ:** Remote হোস্টে GitHub-এর deploy key নেই। `key_file` প্যারামিটার দিয়ে deploy key দিন, বা `accept_hostkey: true` সহ SSH key বসান।
12. **Rolling update অসামঞ্জস্য:** Health check ছাড়া `serial` ব্যবহার করা হয়েছে। প্রতিটি ব্যাচের পর `uri` module দিয়ে `/health` যাচাই করুন এবং ব্যর্থ হলে play থামান।
13. **systemd start ব্যর্থ:** Service ফাইলে `WorkingDirectory` বা `Environment` ভুল, বা Node.js-এর পথ নেই। `journalctl -u app` দিয়ে লগ দেখুন এবং service template ঠিক করুন।
14. **Check মোডে সফল, আসলে ব্যর্থ:** `--check` মোডে `command` / `shell` task আসলে চলে না, তাই তাদের ত্রুটি ধরা পড়ে না। Staging-এ আসল চালনা দিয়ে যাচাই করুন।
15. **Role not found:** Role ভিন্ন `roles_path`-এ ইনস্টল হয়েছে। `ansible-galaxy role list` দিয়ে দেখুন এবং `ansible.cfg`-এ `roles_path` ঠিক করুন।
16. **AWX-এ ব্যর্থ:** AWX execution environment-এ collection বা dependency নেই, বা credential ভিন্ন। Execution environment-এ প্রয়োজনীয় collection যোগ করুন এবং credential যাচাই করুন।
17. **Copy-তে owner root:** `copy` module-এ `owner` ও `group` নির্ধারণ করা হয়নি। `owner: appuser` ও `mode: '0644'` যোগ করুন।
18. **CI-তে Vault ব্যর্থ:** CI-তে Vault পাসওয়ার্ড ফাইল বা secret নেই। CI secret হিসেবে Vault পাসওয়ার্ড দিন এবং `--vault-password-file` ব্যবহার করুন; পাসওয়ার্ড কখনো log-এ ছাপাবেন না।
19. **UFW-তে SSH বিচ্ছিন্ন:** UFW চালু করার আগে SSH পোর্ট (২২) অনুমতি দেওয়া হয়নি। সবসময় প্রথমে `ufw: rule=allow port=22` দিন, তারপর UFW enable করুন।
20. **Playbook ধীর:** Fact সংগ্রহ, SSH সংযোগ ও serial task-এর কারণে ধীর হতে পারে। `gather_facts: false` (যেখানে দরকার নেই), pipelining চালু, `forks` বাড়ানো ও `strategy: free` বিবেচনা করুন।

কেস স্টাডি আলোচনার পয়েন্ট

1. **Vault ফাঁস:** ভুলগুলো — public repository, Vault পাসওয়ার্ড ফাইল commit, `.gitignore`-এ পাসওয়ার্ড ফাইল না রাখা। সঠিক পদ্ধতি: পাসওয়ার্ড password manager-এ রাখা, `--vault-password-file` ব্যবহার করা, `.gitignore`-এ `vault-pass*` যোগ করা, pre-commit hook-এ secret scan (যেমন gitleaks) চালানো। Git history থেকে মুছলেও কপি থাকতে পারে, তাই সব secret rotate করা জরুরি।
2. **ভুল inventory:** নকশাগত ভুল — staging ও production একই ডিরেক্টরিতে, `hosts: all` সহ destructive playbook। নিয়ম: inventory আলাদা ডিরেক্টরি/অ্যাকাউন্টে রাখা, production playbook-এ `hosts: prod`

স্পষ্টভাবে লেখা, `--limit` ও `--check` বাধ্যতামূলক করা, destructive task-এ `vars_prompt` দিয়ে নিশ্চিতকরণ চাওয়া।

3. **Handler বানান:** Ansible notify করা handler না পেলে ত্রুটি দেয় না (নাম মিল না হলে নীরবে বাদ যায়)। প্রতিকার: `ansible-lint` চালানো, handler নামে variable ব্যবহার না করা, deployment-এর পর `uri` বা `command` দিয়ে কনফিগ কার্যকর হয়েছে কি না যাচাই করা।
4. **Non-idempotent seed:** `shell: mongoimport` প্রতিবার ডেটা ঢোকায়। প্রতিকার: `creates` শর্ত, বা আগে ডেটা আছে কি না যাচাই করে `when` দেওয়া; ভালো পদ্ধতি — MongoDB-র `community.mongodb` module বা upsert লজিক ব্যবহার করা।
5. **ভুল tag:** Tag মানুষের ভুলে বদলাতে পারে। প্রতিকার: deployment-এর আগে `ansible-inventory --graph` ও `--list-hosts` দিয়ে হোস্ট তালিকা যাচাই করা; staging ও production আলাদা AWS অ্যাকাউন্ট বা VPC-তে রাখা; tag পরিবর্তনে approval প্রক্রিয়া।
6. **AWX credential:** ভাগ করা অ্যাকাউন্টে audit অসম্ভব। প্রতিকার: প্রত্যেকের নিজের অ্যাকাউন্ট, RBAC দিয়ে team-ভিত্তিক অনুমতি, production job template-এ শুধু senior-দের অনুমতি, job-এ approval ধাপ যোগ করা।
7. **Terraform-Ansible দ্বন্দ্ব:** Hostname Terraform-এর পরিচালিত বৈশিষ্ট্য; Ansible তা বদলানোয় Terraform drift মনে করেছে। প্রতিকার: দায়িত্বের সীমানা মানা, প্রয়োজনে `lifecycle { ignore_changes = [tags] }` ব্যবহার করা, ডেটা EBS volume বা managed ডেটাবেসে রাখা (instance-এর local ডিস্কে নয়)।
8. **Health check ছাড়া rolling update:** সার্ভিস চালু হওয়া মানেই অ্যাপ সুস্থ নয়। প্রতিকার: প্রতিটি ব্যাচের পর `uri` module দিয়ে `/health` যাচাই, `max_fail_percentage: 0` দেওয়া, ব্যর্থ হলে automatic rollback playbook প্রস্তুত রাখা।
9. **Shell injection:** Variable সরাসরি shell-এ বসালে injection সম্ভব। প্রতিকার: `systemd` module ব্যবহার করা (argument আলাদা থাকে), variable যাচাই করা (`assert` module দিয়ে অনুমোদিত মানের তালিকা মেলানো), `shell` এড়িয়ে নির্দিষ্ট module ব্যবহার করা।
10. **Molecule ছাড়া role:** টেস্ট ছাড়া template ভুল production-এ গেছে। প্রতিকার: CI-তে `ansible-lint` ও Molecule বাধ্যতামূলক করা, template বদলের আগে `--check --diff` চালানো, `serial` দিয়ে ছোট ব্যাচে rollout করা।

চূড়ান্ত পরীক্ষার নম্বর বিভাজন (মার্কিং স্কিম)

অংশ	নম্বর	মূল্যায়নের মানদণ্ড
তত্ত্ব (অংশ ১)	৪০	প্রতিটি প্রশ্ন ২ নম্বর — সঠিক ধারণা ১, উদাহরণ/ব্যাখ্যা ১
কাজ ১ — Inventory	৮	সঠিক গ্রুপ ৩, YAML সিনট্যাক্স ২, `--graph` যাচাই ৩
কাজ ২ — Hardening	১২	UFW ৩, fail2ban ৩, root login বন্ধ ৩, unattended-upgrades ৩
কাজ ৩ — MongoDB role	১২	Role কাঠামো ৩, ইনস্টল ও সার্ভিস ৪, Vault পাসওয়ার্ড ৫
কাজ ৪ — Node.js API	১৪	Node.js ৩, git checkout ৩, npm ci ৩, systemd ৩, .env template ২
কাজ ৫ — nginx প্রক্সি	৮	Template ৩, handler ২, health যাচাই ৩
কাজ ৬ — Idempotency	৬	দ্বিতীয় চালনায় সব ok ৪, --check --diff আউটপুট ২

মোট	**১০০**	পাস নম্বর ৬০; ৮০+ হলে Distinction

১৫টি ব্যবহারিক অ্যাসাইনমেন্ট

1. **প্রথম inventory ও ad-hoc:** তিনটি ল্যাব হোস্টের INI ও YAML inventory তৈরি করুন; `ping`, `setup` ও `uptime` ad-hoc কমান্ড চালিয়ে আউটপুট সংরক্ষণ করুন। মূল্যায়ন: *inventory* সিনট্যাক্স সঠিক, সব হোস্ট *reachable*।
2. **প্রথম playbook:** সব হোস্টে `htop` ও `curl` ইনস্টল করার playbook লিখুন; `--check --diff` দিয়ে যাচাই করে চালান। মূল্যায়ন: *idempotent*, দ্বিতীয় চালনায় সব *ok*।
3. **Variable precedence পরীক্ষা:** একই variable-কে role defaults, group_vars, host_vars ও `-e`-তে ভিন্ন মান দিয়ে কোনটি জয়ী হয় তা পরীক্ষা করে টেবিল আকারে লিখুন। মূল্যায়ন: চারটি স্তরের ফল সঠিকভাবে নথিভুক্ত।
4. **Jinja2 template:** nginx virtual host-এর Jinja2 template লিখুন যা `server_name`, `app_port` ও `ssl_enabled` শর্ত অনুযায়ী বদলায়। মূল্যায়ন: তিনটি ভিন্ন *variable* সেটে সঠিক রেন্ডার।
5. **Loop ও conditionals:** পাঁচজন ব্যবহারকারী তৈরি, তিনটি প্যাকেজ ইনস্টল ও দুটি পোর্ট খোলার একটি playbook লিখুন — শুধু Debian হোস্টে। মূল্যায়ন: *loop* ও *when* সঠিক, *Ubuntu*-তে চলে, অন্য *OS*-এ বাদ যায়।
6. **Role তৈরি:** `common` role তৈরি করুন (tasks, handlers, vars, templates, defaults) যা সময় অঞ্চল, NTP ও মৌলিক প্যাকেজ সেট করে। মূল্যায়ন: আদর্শ কাঠামো, পুনঃব্যবহারযোগ্য।
7. **Vault অনুশীলন:** `ansible-vault create` দিয়ে secret ফাইল তৈরি করুন, `encrypt_string` দিয়ে variable এনক্রিপ্ট করুন এবং Vault পাসওয়ার্ড ফাইল ছাড়া playbook চালিয়ে দেখুন কী হয়। মূল্যায়ন: *secret* কোথাও খোলাখুলিভাবে নেই।
8. **Hardening playbook:** SSH hardening, UFW, fail2ban ও unattended-upgrades — একটি সম্পূর্ণ hardening playbook লিখুন ও `--check` দিয়ে যাচাই করুন। মূল্যায়ন: সব নিয়ন্ত্রণ কার্যকর, *SSH* বিচ্ছিন্ন হয় না।
9. **Dynamic inventory:** একটি ক্লাউড বা local inventory plugin দিয়ে dynamic inventory তৈরি করুন ও `--graph` দিয়ে যাচাই করুন। মূল্যায়ন: হোস্ট তালিকা স্বয়ংক্রিয়ভাবে আসে।
10. **MERN ডেপ্লয়মেন্ট:** ল্যাবে MongoDB + Node.js API + nginx — সম্পূর্ণ MERN স্ট্যাক Ansible দিয়ে ডেপ্লয় করুন। মূল্যায়ন: `/health` 200 দেয়, দ্বিতীয় চালনায় *idempotent*।
11. **Rolling update:** `serial: 1` ও `uri` health check-সহ rolling update playbook লিখুন; ইচ্ছাকৃতভাবে ভাঙা কোড দিয়ে দেখুন update থামে কি না। মূল্যায়ন: ভাঙা কোডে *rollout* থামে।
12. **Docker কনটেইনার পরিচালনা:** `community.docker` collection দিয়ে TechNova-র API কনটেইনার চালান ও বন্ধ করুন। মূল্যায়ন: *idempotent*, কনটেইনার পুনরায় তৈরি হয় না।
13. **Molecule টেস্ট:** নিজের একটি role-এর জন্য Molecule scenario লিখুন ও `molecule test` চালিয়ে পাস করান। মূল্যায়ন: *converge* ও *verify* দুটোই সফল।
14. **AWX job:** AWX-এ inventory, credential ও job template তৈরি করে একটি playbook schedule করুন। মূল্যায়ন: *job* সফলভাবে চলে ও *log* সংরক্ষিত হয়।
15. **Terraform + Ansible:** Terraform দিয়ে একটি VM তৈরি করুন, তার output থেকে Ansible inventory তৈরি করে তাতে একটি role চালান। মূল্যায়ন: *Terraform state* ও *Ansible inventory* সামঞ্জস্যপূর্ণ।